

Desarrollo de un pipeline reproducible para la detección y tipificación de cáncer a partir de RNA-Seq

Raúl García Gonzalo

31 de diciembre, 2025

Índice

Importación de las librerías necesarias	1
Importación de los datos	1
Carga de las muestras tumorales y de sus clasificadores	1
Carga de las muestras normales y de sus clasificadores	2
Reducción de las muestras tumorales y restricción de los tipos de cáncer	3
Unión de los datos y filtrado por tipos de cáncer escogidos	5
División del conjunto de datos	6
Análisis exploratorio de los datos	7
Análisis básico	7
Valores faltantes/nulos y valores cero	9
Información de las métricas de los genes	10
Control de calidad de las muestras	12
Detección de <i>outliers</i> con la regla 3*IQR (InterQuantile Range)	13
PCA y detección de <i>outliers</i> con la distancia de Mahalanobis	13
Resumen de correlación	16
Heatmap de los genes	17
Preprocesamiento de los datos	18
Eliminación de los genes sin interés	18
Eliminación de los <i>outliers</i> detectados	18
Normalización y transformación de los datos	19
Eliminación de los genes con baja variabilidad	20
Codificación de las variables clase	21
Selección de características	22
Construcción y evaluación de los modelos	24
Modelos para la predicción del tipo de muestra (normal vs. tumoral)	24
Modelos para la predicción del tipo de cáncer	46
Selección, optimización e interpretabilidad del modelo	58
Modelo de predicción para el tipo de muestra	58
Modelo de predicción para el tipo de cáncer	63
Prueba final del modelo óptimo	71
Modelo para la clasificación del tipo de muestra	71
Modelo para la clasificación del tipo de cáncer	72
Desarrollo de la aplicación web	74

Importación de las librerías necesarias

```
library(data.table)
library(curl)
library(dplyr)
library(kableExtra)
library(caret)
library(knitr)
library(tibble)
library(MASS)
library(ggplot2)
library(tidyr)
library(limma)
library(glmnet)
library(randomForest)
library(e1071)
library(xgboost)
library(keras3)
library(tensorflow)
library(caret)
library(reticulate)
library(class)
library(kknn)
library(lightgbm)
library(MLmetrics)
library(pROC)
library(ranger)
library(lime)
library(iml)
library(tools)
library(shiny)
library(rsconnect)
```

Importación de los datos

Carga de las muestras tumorales y de sus clasificadores

```
set.seed(123)
# Carga de los datos de expresión génica de las muestras tumorales desde GEO.
if (!file.exists("GSM1536837_06_01_15_TCGA_24.tumor_Rsubread_TPM.txt.gz")) {
  curl_download("https://www.ncbi.nlm.nih.gov/geo/download/?acc=GSM1536837&file=GSM1536837_06_01_15_TCGA_24.tumor_Rsubread_TPM.txt.gz")
}

# Se almacenan en un objeto mediante 'fread()'.
tumor_samples <- fread("GSM1536837_06_01_15_TCGA_24.tumor_Rsubread_TPM.txt.gz")

# También se almacenan los clasificadores del tipo de cáncer en un nuevo objeto.
# Este fichero se debe descargar desde el enlace del estudio GSE62944.
tumor_cancer_type_samples <-
  read.table(gzfile("GSE62944_06_01_15_TCGA_24_CancerType_Samples.txt.gz"))

# Se obtienen los nombres de los genes en un nuevo objeto y se eliminan estos
```

```

# de los datos, teniendo así solo los valores de expresión.
genes <- tumor_samples[[1]]
tumor_samples <- as.matrix(tumor_samples[ , -1, drop = FALSE])
# Se introducen estos genes como los nombres de las filas.
rownames(tumor_samples) <- genes
# Para facilitar el trabajo posterior con estos datos, se transpone la matriz y
# se transforma a dataframe.
tumor_samples <- as.data.frame(t(tumor_samples))

# Los clasificadores también se transforman a dataframe. También se le cambian
# los nombres a las columnas para diferenciar la columna con el nombre de las
# muestras con la que contiene los clasificadores del tipo de cáncer.
tumor_cancer_type_samples <- as.data.frame(tumor_cancer_type_samples)
names(tumor_cancer_type_samples) <- c("Samples", "CancerType")
# Se añade una nueva columna con el tipo de muestra. Como todo son muestras
# tumorales, se indica la clase 'Tumor'.
tumor_cancer_type_samples$SampleType <- "Tumor"

```

Carga de las muestras normales y de sus clasificadores

```

# Carga de los datos de expresión génica de las muestras normales desde GEO.
if (!file.exists("GSM1697009_06_01_15_TCGA_24.normal_Rsubread_TPM.txt.gz")) {
  curl_download("https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSM1697009_06_01_15_TCGA_24.normal_
    GSM1697009_06_01_15_TCGA_24.normal_Rsubread_TPM.txt.gz", quiet = FALSE)
}

# Se almacenan en un objeto mediante 'fread()'.
normal_samples <-
  fread("GSM1697009_06_01_15_TCGA_24.normal_Rsubread_TPM.txt.gz")

# También se almacenan los clasificadores del tipo de cáncer en un nuevo objeto.
# Este fichero, como se ha indicado con las muestras tumorales, se debe descargar
# desde el enlace del estudio GSE62944.
normal_cancer_type_samples <- read.table(gzfile("GSE62944_06_01_15_TCGA_24_Normal_CancerType_Samples.tx

# Se obtienen los nombres de los genes en un nuevo objeto y se eliminan estos
# de los datos, teniendo así solo los valores de expresión.
genes <- normal_samples[[1]]
normal_samples <- as.matrix(normal_samples[ , -1, drop = FALSE])
# Se introducen estos genes como los nombres de las filas.
rownames(normal_samples) <- c(genes)
# Para facilitar el trabajo posterior con estos datos, se transpone la matriz y
# se transforma a dataframe.
normal_samples <- as.data.frame(t(normal_samples))

# Los clasificadores también se transforman a dataframe. También se le cambian
# los nombres a las columnas para diferenciar la columna con el nombre de las
# muestras con la que contiene los clasificadores del tipo de origen tisular.
normal_cancer_type_samples <- as.data.frame(normal_cancer_type_samples)
names(normal_cancer_type_samples) <- c("Samples", "CancerType")
# Se añade una nueva columna con el tipo de muestra. Como todo son muestras
# normales, se indica la clase 'Normal'.
normal_cancer_type_samples$SampleType <- "Normal"

```

Reducción de las muestras tumorales y restricción de los tipos de cáncer

Para poder tomar una decisión, primero se han ordenado en orden descendente la cantidad de muestras que se tienen para cada tipo de cáncer tanto en muestras tumorales como en normales. Para ello, se usan las funciones `sort()` y `table()` de manera combinada.

```
# Se ordenan en orden descendente los tipos de cáncer de ambos tipos de datos  
# para observar los conteos y la distribución de las muestras en estos.  
sort(table(tumor_cancer_type_samples[ , 2]), decreasing = TRUE)
```

```
##  
## BRCA UCEC KIRC LUAD LGG THCA HNSC LUSC PRAD COAD SKCM OV STAD BLCA LIHC CESC  
## 1119 554 542 541 532 513 504 502 502 483 472 430 420 414 374 306  
## KIRP LAML GBM READ ACC KICH UCS DLBC  
## 291 178 170 167 79 66 57 48
```

```
sort(table(normal_cancer_type_samples[ , 2]), decreasing = TRUE)
```

```
##  
## BRCA KIRC LUAD THCA PRAD LUSC LIHC HNSC COAD STAD UCEC KIRP KICH BLCA ESCA READ  
## 113 72 59 59 52 51 50 44 41 37 35 32 25 19 13 10  
## CHOL GBM PAAD CESC PCPG SARC THYM SKCM  
## 9 5 4 3 3 2 2 1
```

Se ha decidido trabajar sobre aquellos tipos de cáncer que presenten un número de muestras superior a 500 para las muestras tumorales y de manera simultánea un número de muestras normales superior a 50.

```
# Se genera un nuevo objeto que contiene las clases de tipos de cáncer que  
# contienen 500 o más muestras tumorales. Eso se hace mediante las funciones  
# 'group_by()', 'filter()', y 'ungroup()'.  
cancer_type_above500 <- tumor_cancer_type_samples %>%  
  group_by(CancerType) %>%  
  filter(n() > 500) %>%  
  ungroup()  
  
# De la misma forma que con las muestras tumorales, se genera un nuevo objeto  
# que contiene las clases de tipo de cáncer que contienen, en este caso, por lo  
# menos 50 muestras normales.  
cancer_type_above50 <- normal_cancer_type_samples %>%  
  group_by(CancerType) %>%  
  filter(n() > 50) %>%  
  ungroup()
```

Con las muestras obtenidas en ambos casos, se usa la función `inner.join()` para así obtener solo aquellos clasificadores donde se cumplen estas dos condiciones. Estas se pueden observar en la tabla resultante del chunk, la cual consiste en la variable `cancer_types`.

```
# Se generan unos conteos de ambos tipos de muestras.  
counts_tumor_samples <-  
  cancer_type_above500 %>% count(CancerType, name = "tumor500")  
counts_normal_samples <-  
  cancer_type_above50 %>% count(CancerType, name = "normal50")
```

Y se genera una tabla de formato 'tibble' para observar las clases en las que se cumplen los dos requisitos simultáneamente (500 más muestras tumorales y 50 o más muestras normales). Con los conteos obtenidos previamente se puede la distribución de muestras en estos para cada tipo de cáncer.

```
cancer_types <-
  inner_join(counts_tumor_samples, counts_normal_samples, by = "CancerType") %>%
  arrange(desc(tumor500 + normal50))
```

```
cancer_types
```

```
## # A tibble: 6 x 3
##   CancerType tumor500 normal50
##   <chr>         <int>     <int>
## 1 BRCA          1119       113
## 2 KIRC           542        72
## 3 LUAD           541        59
## 4 THCA           513        59
## 5 PRAD           502        52
## 6 LUSC           502        51
```

Observando el número de muestras tanto tumorales como normales para los tipos de cáncer seleccionados, se ha visto que el clasificador *BRCA* presenta una cantidad de muestras tumorales muy elevada a las demás. Por ende, para equilibrar los grupos se ha decidido eliminar 600 muestras aleatorias de este, de forma que haya unas 500 muestras tumorales para cada origen tisular.

De nuevo mediante la función 'filter()' y añadiendo 'sample()', se eliminan de forma aleatoria 600 muestras de la clase BRCA.

```
cancer_type_above500 <- cancer_type_above500 %>%
  filter(!(CancerType == "BRCA" & Samples %in%
    sample(Samples[CancerType == "BRCA"], 600)))
```

Por último, se ha repetido la tabla de conteos para comprobar y confirmar que se ha llevado a cabo correctamente la eliminación de estos datos.

Se repiten de nuevo los conteos y la tabla 'tibble' mediante 'inner_join()', con el propósito de observar como la distribución ahora se encuentra más equilibrada entre muestras tumorales que previamente.

```
counts_tumor_samples <-
  cancer_type_above500 %>% count(CancerType, name = "tumor500")
cancer_types <-
  inner_join(counts_tumor_samples, counts_normal_samples, by = "CancerType") %>%
  arrange(desc(tumor500 + normal50))
```

```
cancer_types
```

```
## # A tibble: 6 x 3
##   CancerType tumor500 normal50
##   <chr>         <int>     <int>
## 1 BRCA          519       113
## 2 KIRC           542        72
## 3 LUAD           541        59
## 4 THCA           513        59
## 5 PRAD           502        52
## 6 LUSC           502        51
```

Unión de los datos y filtrado por tipos de cáncer escogidos

Con la selección hecha, se ha procedido a unir los datos y los clasificadores. Se ha procedido con estos últimos, los cuales se han unido mediante la función `rbind()`. Después, se han filtrado los clasificadores para obtener solo las muestras de los tipos de cáncer escogidos con `subset()`. Por último se ha observado cuál es la proporción que ocupa cada clasificador (tanto de tipo de muestra como de cáncer) dentro del conjunto de datos.

```
# A partir del objeto con las muestras filtradas que se ha generado previamente,  
# se genera un nuevo objeto que contenga los clasificadores de las muestras  
# normales y tumorales en uno solo.  
samples_cancer_type <- rbind(cancer_type_above500,  
                             cancer_type_above50)  
# Mediante 'subset()' se filtran los clasificadores para obtener solo los  
# seleccionados antes.  
samples_cancer_type <- subset(samples_cancer_type,  
                              CancerType %in% cancer_types$CancerType)  
  
# Para tener una idea de la distribución del tipo de muestras y observar que esta  
# a su vez sea homogénea entre los tipos de cáncer, se genera una tabla de  
# proporciones para ambos casos.  
prop.table(table(samples_cancer_type$SampleType))
```

```
##  
##      Normal      Tumor  
## 0.1151773 0.8848227
```

```
prop.table(table(samples_cancer_type$CancerType))
```

```
##  
##      BRCA      KIRC      LUAD      LUSC      PRAD      THCA  
## 0.1792908 0.1741844 0.1702128 0.1568794 0.1571631 0.1622695
```

Después, se han filtrado las muestras tanto tumorales como normales para mantener solo aquellas de los tipos de cáncer seleccionados anteriormente, y se han combinado de nuevo con `rbind()`.

```
# A partir de los clasificadores que se han filtrado, se obtienen las muestras  
# tumorales cuyos identificadores se han mantenido ya que pertenecen a los tipos  
# de cáncer seleccionados.  
filtered_tumor_samples <-  
  tumor_samples[rownames(tumor_samples) %in% samples_cancer_type$Samples, ]  
  
# Lo mismo se realiza para las muestras normales.  
filtered_normal_samples <-  
  normal_samples[rownames(normal_samples) %in% samples_cancer_type$Samples, ]  
  
# Por último, se unen las muestras tumorales y normales para que se encuentren  
# en un mismo objeto.  
samples <- rbind(filtered_tumor_samples, filtered_normal_samples)
```

División del conjunto de datos

El conjunto de datos se ha dividido en tres subconjuntos: de entrenamiento (*train*), de prueba (*test*) y de validación (*validation*). Primero de todo, se ha dividido el conjunto de datos para obtener un 70% de los

datos de entrenamiento + validación y 30% de este en el conjunto de prueba. Acto seguido, el 70% se ha dividido para que un 85% se destine a los datos del conjunto de entrenamiento y el 15% restante en el de validación.

```
# Se añade una semilla para mantener la reproducibilidad.
set.seed(123)
# Se genera un índice estratificado que selecciona el 70% de las muestras para
# destinarlas a entrenamiento y validación, mientras que el 30% restante se
# destinan al conjunto de prueba.
train_val_index <- createDataPartition(samples_cancer_type$CancerType,
                                       p = 0.70, list = FALSE)[, 1]

# Después, se realiza un nuevo índice estratificado para separar las muestras
# anteriores destinadas a los conjuntos de entrenamiento y validación en un 85%
# de estas hacia el primero y un 15% hacia el segundo.
index <-
  createDataPartition(samples_cancer_type$CancerType[train_val_index],
                      p = 0.85, list = FALSE)[, 1]

# A partir de los índices generados, se construyen los índices finales para
# cada conjunto.
train_index <- train_val_index[index]
validation_index <- train_val_index[-index]
test_index <- setdiff(seq_len(nrow(samples)), train_val_index)

# Con los índices obtenidos, se generan los tres objetos de los conjuntos de
# datos que contienen los clasificadores. A partir de estos, se filtran los
# valores de expresión para obtener aquellos presentes en cada subconjunto.
# Para hacerlo, se usa la función 'match()' y se comprueba que los
# identificadores coincidiesen con el nombre de fila.
train_cancer_type <- samples_cancer_type[train_index, ]
train_samples <- samples[match(train_cancer_type$Samples, rownames(samples)),]

test_cancer_type <- samples_cancer_type[test_index, ]
test_samples <- samples[match(test_cancer_type$Samples, rownames(samples)),]

validation_cancer_type <- samples_cancer_type[validation_index, ]
validation_samples <- samples[match(validation_cancer_type$Samples,
                                    rownames(samples)),]
```

Para comprobar si el reparto ha mantenido correctamente la proporción de los tipos de cáncer, se ha comprobado las proporciones que se muestran en cada uno de los subconjuntos de datos.

```
# Se genera un dataframe para observar las proporciones de cada tipo de cáncer
# en cada conjunto de datos. Se introducen los valores de los tres conjuntos
# generados junto con los del conjunto de datos inicial. De esta forma, se puede
# concluir si la partición se ha realizado de manera homogénea.
proporciones <- data.frame(
  Tipo = names(prop.table(table(samples_cancer_type$CancerType))),
  Inicial = as.vector(prop.table(table(samples_cancer_type$CancerType))),
  Entrenamiento = as.vector(prop.table(table(train_cancer_type$CancerType))),
  Prueba = as.vector(prop.table(table(test_cancer_type$CancerType))),
  Validación = as.vector(prop.table(table(validation_cancer_type$CancerType)))
)

# Se muestra el resultado en formato tabla mediante 'kable()'.
```

```
knitr::kable(proporciones, digits = 4,
             caption = "Proporciones por tipo de cáncer en cada conjunto de datos")
```

Table 1: Proporciones por tipo de cáncer en cada conjunto de datos

Tipo	Inicial	Entrenamiento	Prueba	Validación
BRCA	0.1793	0.1794	0.1791	0.1789
KIRC	0.1742	0.1742	0.1744	0.1734
LUAD	0.1702	0.1699	0.1706	0.1707
LUSC	0.1569	0.1571	0.1564	0.1572
PRAD	0.1572	0.1571	0.1573	0.1572
THCA	0.1623	0.1623	0.1621	0.1626

Análisis exploratorio de los datos

Análisis básico

Primero de todo, se ha hecho un estudio sobre el contenido del conjunto de entrenamiento, observando el número de muestras y genes junto con los conteos y las proporciones que este contiene de cada tipo de muestra y tipo de cáncer.

```
# Para tener una idea del contenido del conjunto de entrenamiento, se genera
# un dataframe con el número de genes analizados junto con el número de muestras
# sanas y tumorales con su proporción dentro del conjunto.
basic_analysis <- data.frame(
  Métrica = c("Número de muestras", "Número de genes analizados",
              "Muestras normales (sanas)", "Proporción de muestras normales",
              "Muestras tumorales", "Proporción de muestras tumorales"),
  Valor = c(format(dim(train_samples)[1], scientific = FALSE),
             format(dim(train_samples)[2], scientific = FALSE),
             format(sum(grepl("Normal", train_cancer_type$SampleType)),
                   scientific = FALSE),
             round(mean(grepl("Normal", train_cancer_type$SampleType)), 4),
             format(sum(grepl("Tumor", train_cancer_type$SampleType)),
                   scientific = FALSE),
             round(mean(grepl("Tumor", train_cancer_type$SampleType)), 4))
)

# Se visualiza el resultado con 'kable()'.
kable(basic_analysis, format = "latex", booktabs = TRUE,
      caption = "Resumen básico del conjunto de entrenamiento") %>%
  kable_styling(latex_options = c("striped", "hold_position"))
```

```
# Se ordenan los tipos de cáncer de forma descendiente teniendo en cuenta la
# cantidad de cada uno que se presentan en el conjunto de entrenamiento. Esto se
# logra mediante las funciones 'sort()' y 'table()'.
cancer_counts <- sort(table(train_cancer_type$CancerType), decreasing = TRUE)
# Se obtienen las proporciones de cada uno y se redondean los valores a 4
# decimales con 'prop.table()' y 'round()'.
cancer_props <- round(prop.table(cancer_counts), 4)

# Se genera un dataframe con la información obtenida anteriormente, incluyendo
```


Table 2: Resumen básico del conjunto de entrenamiento

Métrica	Valor
Número de muestras	2101
Número de genes analizados	23368
Muestras normales (sanas)	236
Proporción de muestras normales	0.1123
Muestras tumorales	1865
Proporción de muestras tumorales	0.8877

```
# el tipo de cáncer, su conteo y su proporción en los datos.
cancer_table <- data.frame(Tipo = names(cancer_counts),
                           Conteo = as.integer(cancer_counts),
                           Proporción = cancer_props)

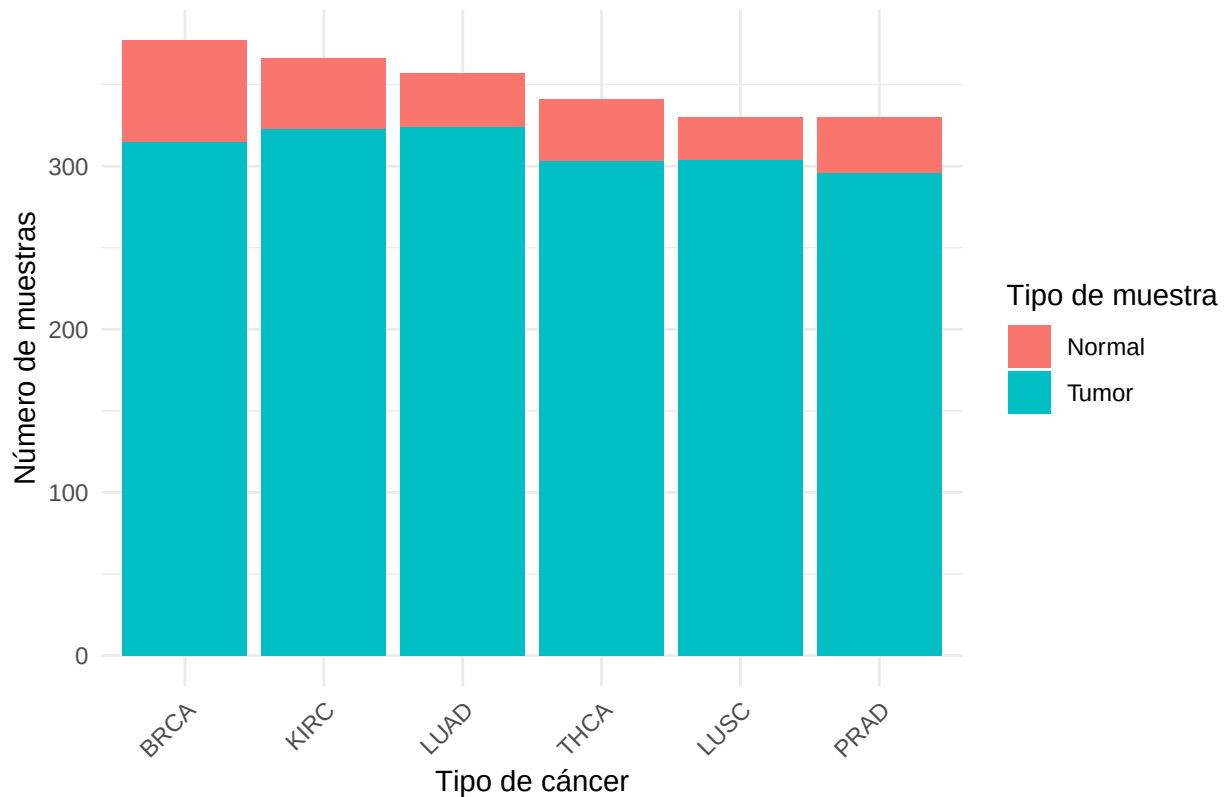
# Se visualiza el resultado mediante una tabla con 'kable()'.
kable(cancer_table, format = "latex", booktabs = TRUE,
      caption = "Distribución de tipos de cáncer en el conjunto de entrenamiento") %>%
  kable_styling(latex_options = c("striped", "hold_position"))
```

Table 3: Distribución de tipos de cáncer en el conjunto de entrenamiento

Tipo	Conteo	Proporción.Var1	Proporción.Freq
BRCA	377	BRCA	0.1794
KIRC	366	KIRC	0.1742
LUAD	357	LUAD	0.1699
THCA	341	THCA	0.1623
LUSC	330	LUSC	0.1571
PRAD	330	PRAD	0.1571

```
# También se genera un gráfico de barras para observar la distribución de tipos
# de muestras en función del tipo de cáncer / origen tisular.
ggplot(train_cancer_type,
       aes(x = reorder(CancerType, -table(CancerType)[CancerType]),
           fill = SampleType)) + geom_bar(position = "stack") +
  labs(x = "Tipo de cáncer", y = "Número de muestras", fill = "Tipo de muestra",
       title = "Distribución Normal vs Tumor por tipo de cáncer") +
  theme_minimal() + theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Distribución Normal vs Tumor por tipo de cáncer



Valores faltantes/nulos y valores cero

Al tratarse de datos de RNA-Seq, estos no deben contener ningún tipo de valor faltante/nulo (NA). Por otro lado, al tratarse de valores de expresión génica sí que se pueden dar valores cero (0.0), dado que estos hacen referencia a la ausencia de expresión.

Junto al recuento de valores faltantes y de valores cero, también se han obtenido los 10 genes y muestras con una cantidad mayor de valores cero del conjunto de datos. La finalidad de observar estos resultados ha sido poder sacar ciertas conclusiones de cómo proceder en el preprocesamiento de los datos con tal de reducir, si se puede hacer sin perder ningún tipo de información relevante para el trabajo, el número de valores cero.

```
# Se cuentan el número de valores faltantes y valores nulos. Además, para los
# valores nulos se obtienen los 10 genes y las 10 muestras que presentan más
# valores nulos. Todo ello se almacena en un objeto con la función 'paste0()'.
na_zeros <- paste0(
  "Presencia de valores nulos o faltantes\n\n", "Valores faltantes: ",
  sum(is.na(train_samples)), "\n", "Valores nulos: ",
  sum(train_samples == 0, na.rm = TRUE), "\n\n",
  "10 genes con más valores nulos:\n", paste(capture.output(print(
    sort(colSums(train_samples == 0, na.rm = TRUE), decreasing = TRUE)[1:10])),
    collapse = "\n"), "\n\n", "10 muestras con más valores nulos:\n",
  paste(capture.output(print(sort(rowSums(train_samples == 0, na.rm = TRUE),
    decreasing = TRUE)[1:10])), collapse = "\n")
)

# Se muestran los resultados obtenidos con 'cat()'.
cat(na_zeros)
```

```
## Presencia de valores nulos o faltantes
```

```
##
## Valores faltantes: 0
## Valores nulos: 7513724
##
## 10 genes con más valores nulos:
##      AMY1B      AMY1C ANKRD20A1 ANKRD20A2      ANXA8L1      APOC2      APOC4      ARL17B
##      2101      2101      2101      2101      2101      2101      2101      2101
##      BAGE2      BAGE3
##      2101      2101
##
## 10 muestras con más valores nulos:
## TCGA-ET-A3BP-01A-21R-A190-07 TCGA-EL-A4K2-01A-11R-A250-07
##                               7380                               6389
## TCGA-ET-A4KN-01A-11R-A250-07 TCGA-33-4566-01A-01R-1443-07
##                               5745                               5613
## TCGA-55-7815-01A-11R-2170-07 TCGA-BH-AODZ-01A-11R-A00Z-07
##                               5457                               5274
## TCGA-AK-3428-01A-02R-1277-07 TCGA-AK-3451-01A-02R-1188-07
##                               5267                               5196
## TCGA-DJ-A2QA-01A-11R-A18C-07 TCGA-E9-A226-01A-21R-A157-07
##                               5196                               5195
```

Información de las métricas de los genes

Se ha generado también una tabla de los 10 primeros genes del conjunto de entrenamiento que contiene la media, la desviación estándar, la varianza y la proporción de valores cero que se presentan por gen. Con ello, se ha podido concluir, entre otras, si se observa un equilibrio entre los valores de expresión de los genes.

```
# Se obtienen las medias, desviaciones estándar, varianzas y la proporción de
# valores zero por gen de todos los genes del conjunto.
mean_eda <- round(colMeans(train_samples, na.rm = TRUE), 4)
sd_eda   <- round(apply(train_samples, 2, sd, na.rm = TRUE), 4)
var_eda  <- round(sd_eda^2, 4)
zero_gen_eda <- round(colSums(train_samples == 0,
                              na.rm = TRUE) / nrow(train_samples), 4)

# Dado que la cantidad de genes es muy elevada, se obtienen los valores de los
# 10 primeros que aparecen en los datos y se almacenan en formato dataframe en
# un nuevo objeto.
gen_information <-
  data.frame(Mean = head(mean_eda, 10), Sd = head(sd_eda, 10),
             Var = head(var_eda, 10), Pct_zero = head(zero_gen_eda, 10))

# Se grafican las métricas de estos 10 genes mediante 'kable()'.
kable(gen_information, caption = "Métricas por gen (primeros 10 genes)") %>%
  kable_styling()
```

Table 4: Métricas por gen (primeros 10 genes)

	Mean	Sd	Var	Pct_zero
1/2-SBSRNA4	2.7254	1.7863	3.1909	0.0000
A1BG	8.3016	11.0253	121.5572	0.0000
A1BG-AS1	1.3732	1.1840	1.4019	0.0010
A1CF	2.2986	6.7312	45.3091	0.1166
A2LD1	6.3943	9.9427	98.8573	0.0000

A2M	577.2753	590.7127	348941.4939	0.0000
A2ML1	7.2054	33.9866	1155.0890	0.0138
A2MP1	0.3688	1.0836	1.1742	0.1009
A4GALT	44.0149	42.9416	1843.9810	0.0000
A4GNT	0.5813	5.3778	28.9207	0.1614

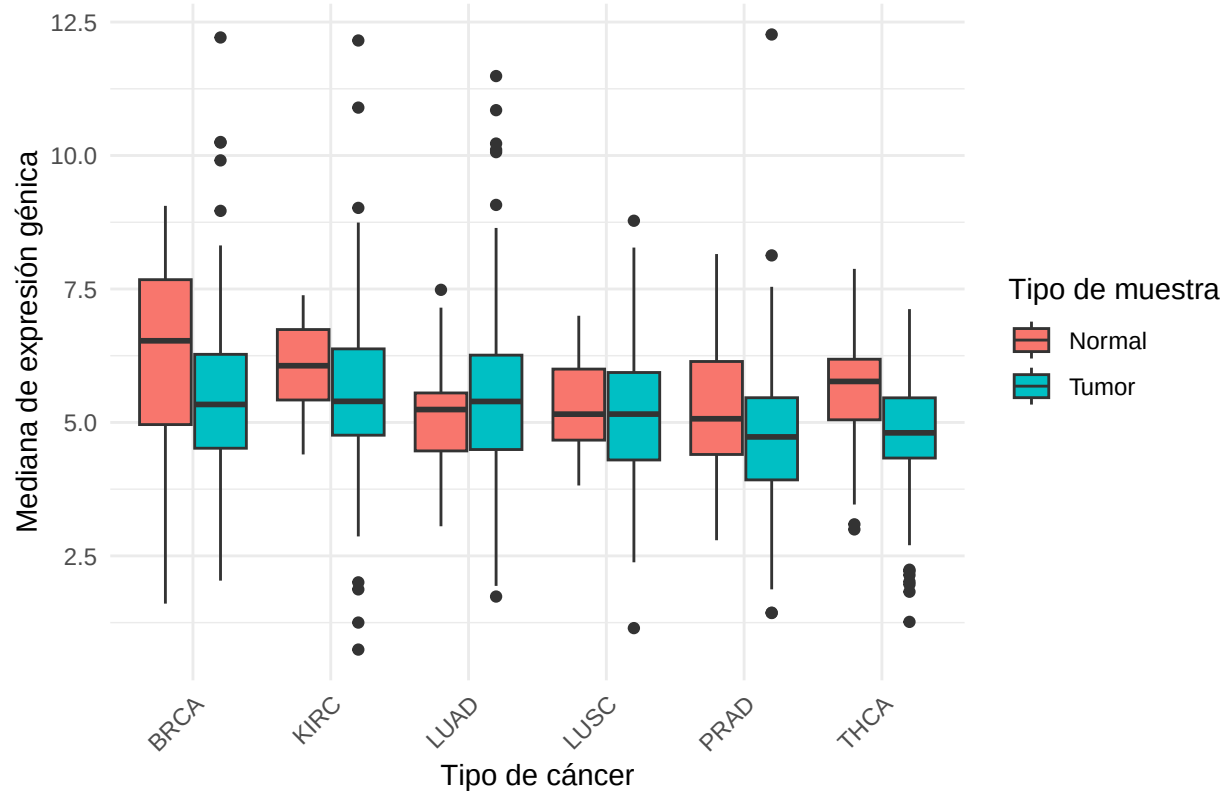
También se ha generado un gráfico de cajas que compara las medianas de expresión génica de las muestras para cada tipo de muestra (normal versus tumoral) y para cada tipo de cáncer. De esta forma, se puede conocer mejor la distribución de los datos.

```
# Se obtiene las medianas de expresión por muestra. Para ello, primero se añade
# la columna 'Sample' del objeto de los clasificadores. Acto seguido, se
# transforma la matriz de expresión a formato largo con 'pivot_longer()',
# teniendo así una fila por gen y muestra. Después, se agrupan los valores
# por muestra y se calcula la mediana de expresión de todos sus genes con
# 'summarise()' y 'median()'. Por último, se desagrupan con 'ungroup()'.
sample_medians <- train_samples %>%
  mutate(Sample = rownames(train_samples)) %>%
  pivot_longer(cols = -Sample, names_to = "Gene", values_to = "Expression") %>%
  group_by(Sample) %>%
  summarise(MedianExpression = median(Expression)) %>%
  ungroup()

# Se añaden las anotaciones de cada muestra mediante el uso de 'left_join()'.
sample_medians <- sample_medians %>%
  left_join(train_cancer_type, by = c("Sample" = "Samples"))

# Se grafican los resultados obtenidos con la función 'ggplot()'.
ggplot(sample_medians, aes(x = CancerType,
  y = MedianExpression, fill = SampleType)) +
  geom_boxplot(position = position_dodge(width = 0.8)) +
  labs(title = "Distribución de la expresión por tipo de cáncer y tipo de muestra",
    x = "Tipo de cáncer", y = "Mediana de expresión génica",
    fill = "Tipo de muestra") + theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Distribución de la expresión por tipo de cáncer y tipo de muestra



Control de calidad de las muestras

Al tratarse de muestras TPM, se espera que la suma de la expresión de los genes sea lo más próxima posible a $1e+6$. Para comprobarlo, se ha generado una tabla donde se muestran las 10 primeras muestras de los datos junto con su valor total de expresión, el número de genes que muestran expresión, la proporción de valores cero que presenta y el tipo de cáncer y de muestra de la que se trata.

```
# Se calculan la expresión total de las muestras, el número de genes expresados
# y la cantidad de ceros que se presentan por muestra.
tot_expr_sample <- rowSums(train_samples, na.rm = TRUE)
expr_gen_sample <- rowSums(train_samples > 0, na.rm = TRUE)
zero_sample <- rowSums(train_samples == 0, na.rm = TRUE) / ncol(train_samples)

# Todo ello se almacena en un dataframe junto con el tipo de cáncer y el tipo
# de muestra de la que se trata.
qc_samples <- data.frame(Sample = rownames(train_samples),
                        ExprTot = tot_expr_sample,
                        GenExpr = expr_gen_sample,
                        PropCero = zero_sample,
                        CancerType = train_cancer_type$CancerType,
                        SampleType = train_cancer_type$SampleType,
                        stringsAsFactors = FALSE)

# Se observan los resultados de las 10 primeras muestras para tener una idea
# de la distribución de los datos, dado que la cantidad de muestras que hay en
# estos es muy extensa.
kable(head(as_tibble(qc_samples)[-1], 10), digits = 4,
      caption = "Control de calidad de las primeras 10 muestras")
```

Table 5: Control de calidad de las primeras 10 muestras

ExprTot	GenExpr	PropCero	CancerType	SampleType
1e+06	18917	0.1905	PRAD	Tumor
1e+06	19785	0.1533	LUAD	Tumor
1e+06	19213	0.1778	PRAD	Tumor
1e+06	19995	0.1443	THCA	Tumor
1e+06	20330	0.1300	BRCA	Tumor
1e+06	20944	0.1037	BRCA	Tumor
1e+06	19448	0.1678	PRAD	Tumor
1e+06	20260	0.1330	KIRC	Tumor
1e+06	20166	0.1370	LUSC	Tumor
1e+06	19897	0.1485	LUAD	Tumor

Detección de *outliers* con la regla 3*IQR (InterQuantile Range)

```
# Se obtiene la expresión total calculada anteriormente, y se almacena en un
# nuevo objeto.
tot <- qc_samples$ExprTot
# Se calculan los cuantiles 1 y 3 a partir de esta expresión con 'quantile()',
# justo antes de obtener el rango intercuartil.
Q1 <- quantile(tot, 0.25)
Q3 <- quantile(tot, 0.75)
IQR <- Q3 - Q1
low_cut <- Q1 - 3 * IQR
high_cut <- Q3 + 3 * IQR
# Se añade al dataframe generado en el control de calidad la columna
# 'OutliersTot', la cual indica mediante FALSE o TRUE si la expresión total se
# encuentra por encima o por debajo de los valores del rango.
qc_samples$OutliersTot <- (tot < low_cut) | (tot > high_cut)

# Se devuelve una salida que contiene el número total de muestras en las que
# la columna 'OutliersTot' devuelve un valor de TRUE; en otras palabras, que
# es considerada un 'outlier'.
cat("\nMuestras 'outlier' por ExprTot (regla 3*IQR):",
    sum(qc_samples$OutliersTot), "\n")
```

```
##
## Muestras 'outlier' por ExprTot (regla 3*IQR): 0
```

PCA y detección de *outliers* con la distancia de Mahalanobis

También se ha realizado un estudio con PCA con el propósito de observar cómo explican la varianza sus 3 primeros componentes (PC1, PC2 y PC3).

```
# Se obtienen los genes más variables.
n_variables <- min(500, ncol(train_samples))
variable_genes <- names(sort(var_eda, decreasing = TRUE))[1:n_variables]

# Se genera un estudio PCA sobre los genes más variables (con escalado y centrado).
k_pca <- min(500, length(variable_genes))
if (k_pca >= 3) {
  X <- scale(train_samples[, variable_genes[1:k_pca], drop = FALSE],
```

```

        center = TRUE, scale = TRUE)
X <- X[complete.cases(X), , drop = FALSE]
pca <- prcomp(X, center = FALSE, scale. = FALSE)

# Se obtiene el valor que indica cuánto explican la varianza los primeros 3
# componentes de PCA, y se devuelve como salida.
pc_var <- summary(pca)$importance[2, 1:3]
cat("\nVarianza PCA explicada por PC1-PC3:", round(pc_var, 4), "\n")

# Se obtienen los 'score' de los 3 componentes y se guardan en un nuevo objeto
# en formato data.frame.
pca_scores <- as.data.frame(pca$x[, 1:3])
# También se crea una columna con el nombre de las muestras, otra con el tipo
# de cáncer y una final con el tipo de muestra.
pca_scores$Sample <- rownames(pca_scores)
pca_scores$CancerType <-
  train_cancer_type$CancerType[match(pca_scores$Sample,
                                     train_cancer_type$Samples)]

pca_scores$SampleType <-
  train_cancer_type$SampleType[match(pca_scores$Sample,
                                     train_cancer_type$Samples)]
} else {
  pca <- NULL; pca_scores <- NULL
  cat("No hay suficientes genes para poder calcular PCs",
      "(se necesita un mínimo de 3 genes).\n")
}

```

```

##
## Varianza PCA explicada por PC1-PC3: 0.0956 0.0644 0.05

```

Sobre los resultados, a su vez, se ha calculado la distancia de Mahalanobis. Con esta, ha sido posible detectar la presencia de ciertos *outliers* que se han obtenido en el conjunto de entrenamiento, los cuales se eliminarán posteriormente durante el preprocesamiento.

```

# Distancia de Mahalanobis en PCA (detección de outliers multivariados).

# - Se seleccionan los primeros PCs (máximo 5 o los disponibles) para capturar # la mayor variabilidad.

# Se seleccionan los primeros PCs, y se calcula la distancia de Mahalanobis de
# cada muestra respecto al centro multivariante definido por estos mediante el
# uso de la función 'mahalanobis()'.
kpc <- min(5, ncol(pca$x))
md <- mahalanobis(pca$x[, 1:kpc], center = colMeans(pca$x[, 1:kpc]),
                 cov = cov(pca$x[, 1:kpc]))
# Se obtienen un p-value para cada muestra mediante 'pchisq()'.
pvals_md <- pchisq(md, df = kpc, lower.tail = FALSE)
# Se añaden tanto las distancias de Mahalanobis como el p-value obtenido al
# data.frame de los scores de PCA.
pca_scores$MD <- md
pca_scores$MD_val <- pvals_md
# Se añade también una columna que indica mediante 'TRUE' si una muestra es
# considerada o no 'outlier', siendo así cuando el p-value es menor a 0.001.
pca_scores$Outlier_MD <- pvals_md < 0.001

# Aquellas muestras identificadas como 'outlier' se extraen y se almacenan en
# un nuevo objeto.

```

```
outlier_samples <- pca_scores$Sample[pca_scores$Outlier_MD]

# Se presenta en pantalla el número de valores atípicos detectados y su
# identificador de muestra.
message("Outliers detectados con Mahalanobis sobre los PCs: ",
        length(outlier_samples))
```

```
## Outliers detectados con Mahalanobis sobre los PCs: 15
```

```
message("\nListado de los outliers detectados:")
```

```
##
## Listado de los outliers detectados:
```

```
outlier_samples
```

```
## [1] "TCGA-H9-A6BY-01A-11R-A30B-07" "TCGA-79-5596-01A-31R-1949-07"
## [3] "TCGA-J4-A67K-01A-21R-A30B-07" "TCGA-KK-A7AW-01A-11R-A320-07"
## [5] "TCGA-49-4506-01A-01R-1206-07" "TCGA-43-6771-01A-11R-1820-07"
## [7] "TCGA-QU-A6IN-01A-11R-A31N-07" "TCGA-90-7769-01A-11R-2125-07"
## [9] "TCGA-M7-A722-01A-12R-A36G-07" "TCGA-33-A5GW-01A-11R-A27Q-07"
## [11] "TCGA-B0-5702-01A-11R-1541-07" "TCGA-AK-3447-01A-01R-1766-07"
## [13] "TCGA-18-3412-01A-01R-0980-07" "TCGA-64-1678-01A-01R-0946-07"
## [15] "TCGA-33-4583-01A-01R-1443-07"
```

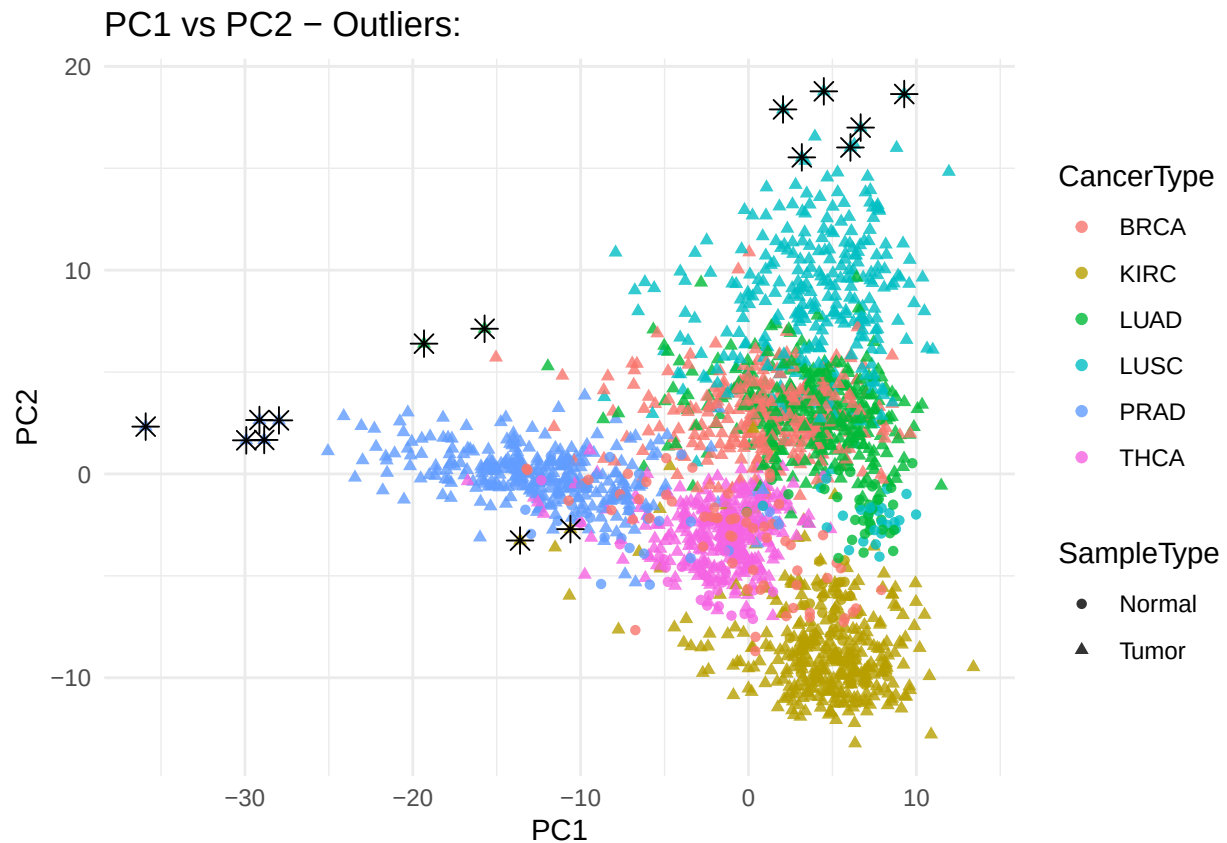
```
# Gráfico de PCA.
```

```
# Se genera una nueva columna que, en caso de que la columna 'Outlier_MD' sea
# 'TRUE', devuelve el valor 'Outlier'; en caso de que sea 'FALSE', devuelve
# el texto 'Influer'.
```

```
pca_scores$OutlierLabel <- ifelse(pca_scores$Outlier_MD, "Outlier", "Influer")
```

```
# Se genera un gráfico de los dos primeros componentes, indicando por color los
# diferentes tipos de cáncer, por forma el tipo de muestra y, en caso de que se
# trate de valores atípicos, se remarca con un símbolo en negro.
```

```
ggplot(pca_scores, aes(x = PC1, y = PC2, color = CancerType,
                        shape = SampleType)) + geom_point(alpha = 0.8) +
  geom_point(data = subset(pca_scores, OutlierLabel == "Outlier"),
            aes(x = PC1, y = PC2), color = "black", shape = 8, size = 3) +
  theme_minimal() + labs(title = "PC1 vs PC2 - Outliers: ",
                        sum(pca_scores$Outlier_MD))
```

Resumen de correlación

Se ha calculado la correlación media absoluta de los 50 genes más variables con el propósito de poder decidir si se observaba cierta correlación entre los genes del conjunto de datos.

```
# Se seleccionan los 50 genes más variables. También se genera un objeto vacío
# que contendrá los resúmenes de correlación.
variable50_genes <- variable_genes[1:min(50, length(variable_genes))]
corr_summary <- NULL

# Mediante el uso de 'if' se comprueba que haya más de 3 genes variables.
if (length(variable50_genes) >= 3) {
  # Se calcula la matriz de correlación y la media absoluta entre pares de genes
  # mediante el uso de las funciones 'cor()' y 'mean()'.
  corr_matrix <- cor(train_samples[, variable50_genes],
                     use = "pairwise.complete.obs")
  abs_mean_corr <- mean(abs(corr_matrix[upper.tri(corr_matrix)]))
  # Se almacena la media absoluta en formato de lista.
  corr_summary <- list(abs_mean_corr = abs_mean_corr,
                      variable50_genes = variable50_genes)

  # Se presenta en pantalla el valor de correlación absoluta obtenido.
  cat("Correlación media absoluta de los 50 genes más variables:",
      round(abs_mean_corr, 3), "\n")
}
```

```
## Correlación media absoluta de los 50 genes más variables: 0.081
```

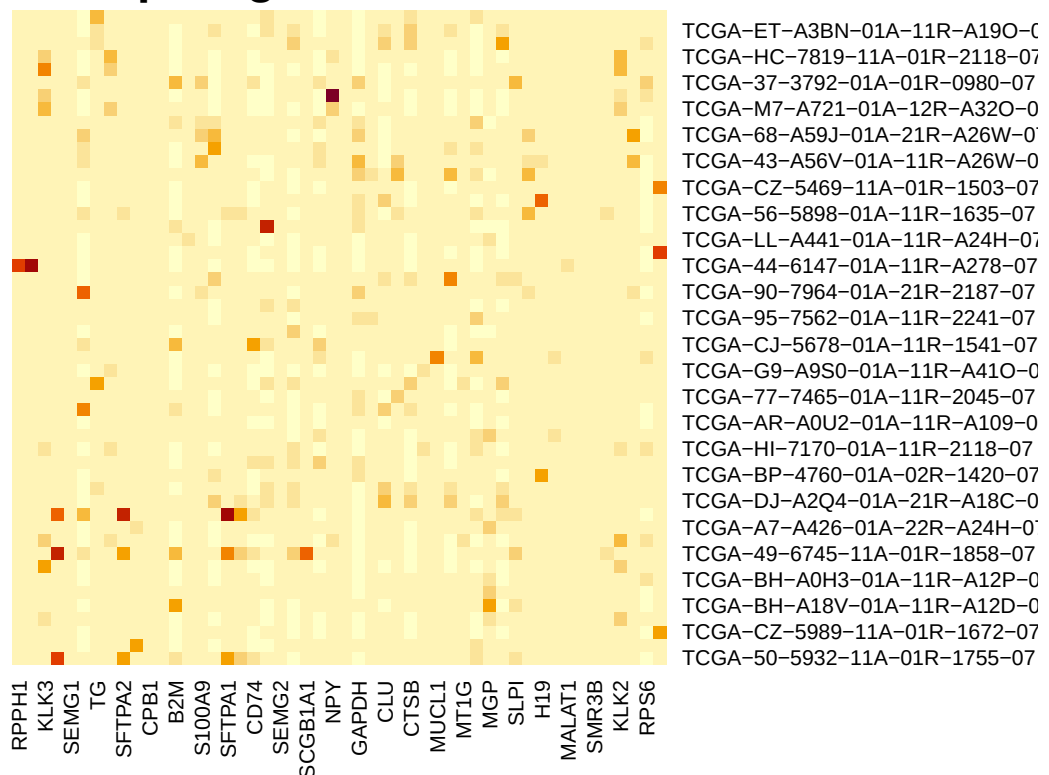
Heatmap de los genes

```
# Se genera un objeto vacío que contendrá la matriz.
matrix <- NULL
# Mediante 'if' se comprueba que haya más de dos genes variables.
if (length(variable50_genes) >= 2) {
  # Se introducen los valores de expresión de los genes más variables y se
  # escalan los datos con 'scale()'.
  matrix <- train_samples[, variable50_genes, drop = FALSE]
  matrix <- scale(matrix)

  # En caso de que hay más de 50 muestras, se toma un muestra aleatoria con
  # 'sample()' de 50 de ellas para generar el heatmap.
  if (nrow(matrix) > 50) mat_plot <- matrix[sample(nrow(matrix), 50), ]
  else mat_plot <- matrix
  heatmap_matrix <- mat_plot

  # Se grafica el heatmap sin reordenar filas ni columnas.
  heatmap(mat_plot, Colv = NA, Rowv = NA, scale = "none",
    main = "Heatmap 50 genes más variables")
}
```

Heatmap 50 genes más variables



Preprocesamiento de los datos

Eliminación de los genes sin interés

Con el análisis exploratorio de los datos se ha visto que había ciertos genes en el conjunto de datos que contenían todo valores cero; en otras palabras, no contenían ningún tipo de interés para el trabajo ya que mostraban una ausencia de expresión para todas las muestras. Por ello, se ha decidido eliminar estos genes de los tres conjuntos de datos.

```
# Función para eliminar genes sin interés de los datos.
genes_wo_int <- function(data, num_zero) {
  # En caso de que el segundo argumento introducido sea un número de longitud 1.
  if (!is.numeric(num_zero) || length(num_zero) != 1)
    stop("'num_zero' debe ser un número y de longitud 1.")

  # Se cuenta el número de ceros por cada columna (es decir, por gen).
  zeros <- colSums(data == 0, na.rm = TRUE)
  # Se marcan los genes que tienen un número de ceros igual o superior al
# segundo argumento mediante 'TRUE' y se eliminan de los datos.
  condition <- zeros >= as.integer(num_zero)
  filtered_data <- data[, !condition, drop = FALSE]
  # Se muestra un mensaje en pantalla con el número de genes inicial, los
# eliminados y los genes que han quedado en el conjunto.
  message(sprintf("Genes antes: %d | Eliminados: %d | Genes después: %d",
                  ncol(data), sum(condition), ncol(filtered_data)))
  # Se obtiene como resultado un nuevo objeto con los datos filtrados
  return(filtered_data)
}
```

```
# Eliminación de los genes de interés del conjunto de entrenamiento.
train_samples <- genes_wo_int(train_samples, nrow(train_samples))
```

```
## Genes antes: 23368 | Eliminados: 532 | Genes después: 22836
```

```
# Se obtiene un listado de los genes de interés del conjunto de entrenamiento.
genes_int <- colnames(train_samples)

# Se eliminan los genes sin interés de los conjuntos de prueba y validación.
test_samples <- test_samples[, genes_int, drop = FALSE]
validation_samples <- validation_samples[, genes_int, drop = FALSE]
```

Eliminación de los *outliers* detectados

También en el EDA se han detectado ciertos *outliers* en el conjunto de datos, por lo que estas muestran también se han eliminado mediante la generación de la función `outliers_datos` tanto en el conjunto de datos como en el de los clasificadores.

```
# Se identifican los 'outliers' dentro del objeto de valores de expresión y del
# de los clasificadores.
outliers_in_samples <- outlier_samples %in% rownames(train_samples)
present_in_classif <-
  outlier_samples %in% as.character(train_cancer_type$Samples)

outliers_datos <- outlier_samples[outliers_in_samples]
```

```

# Se comprueba con un 'if' que haya por lo menos 1 'outlier'.
if (length(outliers_datos) > 0) {
  # Se eliminan estos de los datos de expresión y se devuelve un mensaje con
  # el número de filas eliminadas.
  train_samples <- train_samples[!rownames(train_samples) %in% outliers_datos,]
  message("Eliminadas ", length(outliers_datos),
    " filas de outliers de 'train_samples'.")
} else {
  # En caso que no haya valores atípicos, se imprime un mensaje indicando que no
  # se eliminó ninguna fila.
  message("No se eliminaron filas de 'train_samples'.")
}

```

Eliminadas 15 filas de outliers de 'train_samples'.

```

# Se realiza el mismo proceso acabo de describir, pero esta vez para los
# clasificadores de las muestras.
outliers_classif <- outlier_samples[present_in_classif]
if (length(outliers_classif) > 0) {
  train_cancer_type <-
    train_cancer_type[!as.character(train_cancer_type$Samples) %in%
      outliers_classif,]
  message("Eliminadas ", length(outliers_classif),
    " filas de outliers de 'train_cancer_type'.")
} else {
  message("No se eliminaron filas de 'train_cancer_type'.")
}

```

Eliminadas 15 filas de outliers de 'train_cancer_type'.

Normalización y transformación de los datos

Tras haber eliminado los genes prescindibles y los *outliers*, es necesario comprobar que los datos se encuentran normalizados (siendo datos TPM deberían presentar una normalización, siendo el valor de la expresión de cada muestra $1e+6$).

A su vez, se ha observado que los valores de expresión de los genes no se encuentran equilibrados, lo que dificulta la comparativa entre ellos. Con tal de mitigar este problema, se han transformado los datos mediante la forma $\log_2(x + 1)$.

Para ello, se ha generado la función `norm_transf`.

```

# Función para comprobar la normalización TPM y transformar los datos con
# log2 (x + 1).
norm_transf <- function(data, tpm = TRUE, message = TRUE) {
  # Primero, se comprueba que la mediana de la suma de cada una de las filas
  # (las muestras) sea lo más similar a 1e6 posible, teniendo en cuenta que se
  # puede presentar un pequeño error.
  if (tpm) {
    rs_med <- median(rowSums(data, na.rm = TRUE), na.rm = TRUE)
    pct_diff <- (rs_med - 1e6) / 1e6

    # Se devuelve en mensaje con la mediana de la suma de las filas y la
    # diferencia relativa respecto al valor referencia, 1e6.
    if (message) {
      message("Mediana rowSums: ", rs_med, "\n",

```

```

        "Diferencia relativa respecto a 1e6: ", round(pct_diff * 100, 2), "%")
    }
}

# Después, se transforman los datos con log2(x+1).
transformed_data <- log2(data + 1)

# Se devuelve como resultado de la función los datos transformados.
return(transformed_data)
}

```

```

# Transformación de los conjuntos de datos.
train_samples <- norm_transf(train_samples)

```

```

## Mediana rowSums: 1e+06
## Diferencia relativa respecto a 1e6: 0%

```

```

test_samples <- norm_transf(test_samples)

```

```

## Mediana rowSums: 1e+06
## Diferencia relativa respecto a 1e6: 0%

```

```

validation_samples <- norm_transf(validation_samples)

```

```

## Mediana rowSums: 1e+06
## Diferencia relativa respecto a 1e6: 0%

```

Eliminación de los genes con baja variabilidad

Con los datos transformados, se ha procedido a la eliminación de los genes con una baja variabilidad.

Primero, se han obtenido las varianzas de los genes del conjunto de entrenamiento y se ha generado un resumen estadístico para observar la distribución de estos.

```

# Se calcula la varianza de cada gen.
gene_var <- apply(as.matrix(train_samples), 2, var, na.rm = TRUE)

# Se muestra un resumen mediante 'summary()' de las varianzas.
summary(gene_var)

```

```

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0000  0.2417  0.4585  0.8601  1.0224 26.0903

```

A partir de esta distribución, se ha escogido un valor de *threshold* para filtrar aquellos datos que muestran una variabilidad menor a este. Los genes eliminados se han guardado en un fichero CSV de salida denominado `removed_genes_low_var.csv`, con el propósito de así poder mantener la reproducibilidad del trabajo.

```

# Se introduce el threshold deseado.
threshold <- 0.25

# Se separan en dos objetos los genes que se quieren mantener (aquellos con una
# varianza superior o igual al threshold) de aquellos que no (varianza inferior
# al threshold).

```

```

keep_genes <- names(gene_var)[gene_var >= threshold]
removed_genes <- names(gene_var)[gene_var < threshold]

# Se mantienen los genes con una varianza igual o superior, eliminando los demás.
train_samples <- train_samples[, keep_genes, drop = FALSE]

# Con tal de mantener la reproducibilidad, se genera un fichero CSV que contiene
# los genes con baja varianza eliminados.
write.csv(data.frame(gene = removed_genes),
          file = "removed_genes_low_var.csv", row.names = FALSE)

# Para finalizar, se imprime un mensaje con el número de genes iniciales,
# los mantenidos y los eliminados.
message("Genes iniciales: ", length(gene_var), "\n", "Genes mantenidos: ",
        length(keep_genes), "\n", "Genes eliminados: ", length(removed_genes))

## Genes iniciales: 22836
## Genes mantenidos: 16901
## Genes eliminados: 5935

```

Seguidamente, se han eliminado estos mismos genes en los conjuntos de prueba y de validación.

```

# Se obtiene un listado de los genes mantenidos en el conjunto de entrenamiento.
maintained_genes <- colnames(train_samples)

# Se hace un filtrado con este listado sobre los conjunto de prueba y validación,
# eliminando así los genes con baja varianza.
test_samples <- test_samples[, maintained_genes, drop = FALSE]
validation_samples <- validation_samples[, maintained_genes, drop = FALSE]

```

Codificación de las variables clase

Para facilitar el trabajar con los datos, los clasificadores se han convertido a factores. Para ello, se ha generado la función `convert_classifiers`. Las clases de `CancerType` se han pasado simplemente a factor manteniendo los identificadores de cada uno de ellos, mientras que los de `SampleType` se han convertido posteriormente a numérico para poder trabajar de forma más simple con estos.

```

# Función para convertir los clasificadores a factores.
convert_classifiers <- function(data) {
  # Se transforman a factor los clasificadores del tipo de cáncer, manteniendo
  # como etiquetas las siglas que diferencian a cada uno de ellos y que indican
  # el tipo de cáncer / origen tisural al que pertenecen.
  data$CancerType <-
    factor(data$CancerType, levels = c("BRCA", "KIRC", "LUAD", "LUSC",
                                       "PRAD", "THCA"))

  # Se transforman a factor los clasificadores del tipo de muestra, definiendo
  # como niveles 'Normal' y 'Tumor'. Después, se convierten en entero con la
  # función 'as.integer()' para facilitar el trabajo con los modelos predictivos.
  data$SampleType <- as.integer(factor(data$SampleType,
                                       levels = c("Normal", "Tumor")))

  # Se devuelve de nuevo el fichero inicial, pero con los clasificadores ya
  # codificados.
  return(data)
}

```

```
# Cambio de los clasificadores de los subconjuntos de datos.
train_cancer_type <- convert_classifiers(train_cancer_type)
test_cancer_type <- convert_classifiers(test_cancer_type)
validation_cancer_type <- convert_classifiers(validation_cancer_type)
```

Selección de características

Para poder observar aquellos genes que presentan diferencias significativas, se ha procedido con un análisis de expresión diferencial mediante **limma**. Primero se ha obtenido un panel en relación al tipo de muestra, y se ha hecho lo mismo después para el tipo de cáncer. De estos paneles se han obtenidos los genes únicos (que no se encuentran repetidos), de manera que se ha obtenido el número total de genes que presentan diferencias significativas para ambos casos.

```
# Generación de un código con limma para el análisis de expresión diferencial
# para el tipo de muestra.

# Primero, se transforman los datos en matriz y se transponen con 'as.matrix()'
# y 't()'. Por otro lado, se genera un nuevo objeto con los clasificadores de
# estos.
limma_samples <- t(as.matrix(train_samples))
limma_cancer_type <- train_cancer_type

# Se transforman los clasificadores del tipo de muestra a factor, indicando las
# etiquetas 'Normal' y 'Tumor'.
limma_cancer_type$SampleType <- factor(as.vector(limma_cancer_type$SampleType),
                                     levels = c(1, 2),
                                     labels = c("Normal", "Tumor"))

# Se genera un diseño con 'model.matrix()', a partir del cual se crea un
# contraste con 'makeContrasts()'.
design_st <- model.matrix(~ 0 + SampleType, data = limma_cancer_type)
contrasts_st <- makeContrasts(SampleTypeTumor - SampleTypeNormal,
                             levels = design_st)

# Se realiza un ajuste del modelo con 'lmFit()' y 'contrasts.fit()', y se lleva
# a cabo un cálculo de estadísticos con 'eBayes()'.
fit_st <- lmFit(limma_samples, design_st)
fit_st2 <- contrasts.fit(fit_st, contrasts_st)
fit_st2 <- eBayes(fit_st2)

# Se genera una lista vacía que contendrá los genes seleccionados.
genes_limma_st <- list()
# Se seleccionan los 50 genes más significativos para cada contraste.
for (coef_name in colnames(contrasts_st)) {
  genes_limma_st[[coef_name]] <- topTable(fit_st2, coef = coef_name,
                                          number = 50, sort.by = "p")
}

# Para finalizar, se genera un panel de genes diferencialmente expresados para
# el tipo de muestra.
panel_limma_st <- unique(unlist(lapply(genes_limma_st, rownames)))

# Generación de un código con limma para el análisis de expresión diferencial
# para el tipo de cáncer.
```

```

# De la misma forma que con el tipo de muestra, ahora se genera un diseño para
# comparar todos los tipos de cáncer entre sí.
design_limma <- model.matrix(~ 0 + CancerType, data = limma_cancer_type)

# Se añaden los nombres de fila y de columna.
colnames(design_limma) <- make.names(colnames(design_limma))
rownames(design_limma) <- colnames(limma_samples)

# Y se almacenan también los niveles en un nuevo objeto.
levels_ct <- levels(limma_cancer_type$CancerType)
# Se generan los contrastes por pares entre los tipos de cáncer.
contrast_vector <- apply(combn(levels_ct, 2), 2,
                          function(x) paste0("CancerType", x[1], "-",
                                                "CancerType", x[2]))
contrasts <- makeContrasts(contrasts = contrast_vector, levels = design_limma)

# Se realiza el ajuste del modelo y la obtención de los estadísticos.
fit <- lmFit(limma_samples, design_limma)
fit2 <- contrasts.fit(fit, contrasts)
fit2 <- eBayes(fit2)

# Se genera una nueva lista vacía para almacenar los genes seleccionados.
genes_limma_ct <- list()
# Y de nuevo se seleccionan los 50 genes más significativos para cada contraste.
for (coef_name in colnames(contrasts)) {
  genes_limma_ct[[coef_name]] <- topTable(fit2, coef = coef_name,
                                           number = 50, sort.by = "p")
}

# Se genera el panel de genes diferencialmente expresados para el tipo de cáncer.
panel_limma_ct <- unique(unlist(lapply(genes_limma_ct, rownames)))

# A partir de los dos paneles, se genera uno de único que contiene los genes
# obtenidos en estos. Esto se consigue con la función 'union()'.
panel_limma <- union(panel_limma_st, panel_limma_ct)

# Para finalizar, se imprime un mensaje con el número de genes que presenta este
# último panel generado.
message("Genes con diferencias significativas según el análisis de expresión
        diferencial con limma: ", length(panel_limma), "\n")

```

```

## Genes con diferencias significativas según el análisis de expresión
##          diferencial con limma: 421

```

A partir de los genes significativos identificados en **limma** se ha realizado un filtrado sobre los tres subconjuntos de datos, de manera que solo se mantengan estos.

```

# Se filtran los conjuntos de datos para que estos contengan solamente los genes
# seleccionados con limma.
train_samples <- train_samples[, panel_limma]
test_samples <- test_samples[, panel_limma]
validation_samples <- validation_samples[, panel_limma]

```


Construcción y evaluación de los modelos

Esta parte se ha dividido en dos: primero de todo, una generación de modelos con el propósito de predecir y clasificar muestras normales o tumorales. Con esta hecha, y sobre las muestras tumorales, se han generado nuevos modelos que sean capaces de clasificar cada muestra tumoral por el tipo de cáncer al que pertenecen.

Modelos para la predicción del tipo de muestra (normal vs. tumoral)

Generación de los modelos

Antes de empezar con la generación de modelos, se ha creado una función para obtener las métricas o estadísticos de todos los modelos de una forma más sencilla y eficaz. Partiendo de la matriz de confusión, se obtienen la precisión tanto de cada clase como global, el *recall* y el *F1 score* de la clase minoritaria y del global, el valor *Kappa*, la sensibilidad, la especificidad y la *accuracy* balanceada. Junto con el objeto de la matriz de confusión, también se debe introducir el nombre del modelo para poder comparar cada uno de ellos posteriormente. Las métricas de cada uno se almacenan en forma de *dataframe*.

```
metrics_samples <- function(cm, model_name = "Modelo") {  
  # Se extraen las celdas de la matriz de confusión.  
  predN_refN <- cm$table[1, 1]  
  predN_refT <- cm$table[1, 2]  
  predT_refN <- cm$table[2, 1]  
  predT_refT <- cm$table[2, 2]  
  
  # Se indican los positivos reales, falsos positivos, falsos negativos y  
  # negativos reales teniendo en cuenta 'Normal' como la clase positiva.  
  TP_n <- predN_refN  
  FP_n <- predN_refT  
  FN_n <- predT_refN  
  TN_n <- predT_refT  
  
  # Y se obtienen las métricas de precisión, 'recall' (sensibilidad), el ratio  
  # de falsos positivos y falsos negativos y el valor F1. Los resultados se  
  # redondean a 4 decimales.  
  precision_n <- round(TP_n / (TP_n + FP_n), 4)  
  recall_n <- round(TP_n / (TP_n + FN_n), 4)  
  fpr_n <- round(FP_n / (FP_n + TN_n), 4)  
  fnr_n <- round(FN_n / (FN_n + TP_n), 4)  
  f1_n <- round(2 * precision_n * recall_n / (precision_n + recall_n), 4)  
  
  # Se realiza el mismo procedimiento, pero esta vez indicando 'Tumor' como la  
  # clase positiva.  
  TP_t <- predT_refT  
  FP_t <- predT_refN  
  FN_t <- predN_refT  
  TN_t <- predN_refN  
  
  precision_t <- round(TP_t / (TP_t + FP_t), 4)  
  recall_t <- round(TP_t / (TP_t + FN_t), 4)  
  fpr_t <- round(FP_t / (FP_t + TN_t), 4)  
  fnr_t <- round(FN_t / (FN_t + TP_t), 4)  
  f1_t <- round(2 * precision_t * recall_t / (precision_t + recall_t), 4)  
  
  # A partir de los resultados de ambos clasificadores, se calculan los valores  
  # globales de las métricas haciendo la media de ambos.  
  precision <- round((precision_n + precision_t) / 2, 4)
```

```

recall <- round((recall_n + recall_t) / 2, 4)
fpr <- round((fpr_n + fpr_t) / 2, 4)
fnr <- round((fnr_n + fnr_t) / 2, 4)
f1_macro <- round((f1_n + f1_t) / 2, 4)

# Por otro lado, se obtiene la 'accuracy' (exactitud), el coeficiente Kappa,
# la especificidad y la exactitud balanceada directamente de los estadísticos
# de la matriz de confusión.
accuracy <- round(unname(cm$overall["Accuracy"]), 4)
kappa <- round(unname(cm$overall["Kappa"]), 4)
specificity <- round(unname(cm$byClass["Specificity"]), 4)
bal_accuracy <- round(unname(cm$byClass["Balanced Accuracy"]), 4)

# Se almacenan las métricas que se han considerado más importante para la
# comparación de modelos en formato dataframe.
model_metrics <-
  data.frame("Model" = model_name, "Prec_n" = precision_n,
            "Prec_t" = precision_t, "Precision" = precision,
            "Recall_n" = recall_n, "Recall" = recall,
            "F1_n" = f1_n, "F1_macro" = f1_macro,
            "Kappa" = kappa, "Spec" = specificity, "Bal_acc" = bal_accuracy)
}

```

También se han preparado los datos. Para ello, los conjuntos que contienen los valores de expresión se han convertido a matriz, mientras que las clases se han mantenido idénticas a las originales, pero almacenando la información en nuevos objetos.

```

# Se preparan los conjuntos de entrenamiento y de validación para la generación
# de los modelos. Para ello, los objetos que contienen los valores de expresión
# se transforman a matriz.
# En el caso del conjunto de entrenamiento, se genera un nuevo objeto con los
# identificadores del tipo de muestra. Para el conjunto de validación, se
# realiza lo mismo pero indicando que cuando el valor sea 2 ('Tumor') en los
# datos originales se cambie a 1, mientras que en caso contrario (es decir,
# 'Normal') ahora pase a 0.
samples_train <- as.matrix(train_samples)
train_type <- train_cancer_type$SampleType
samples_val <- as.matrix(validation_samples)
validation_type <- factor(ifelse(validation_cancer_type$SampleType == 2, 1, 0))

```

Con la función para obtener los estadísticos y los datos preparados, se ha procedido a la generación de los modelos. En este caso, al tratarse de la clasificación de muestras en normales o tumorales, se han generado solo modelos predictivos binarios. En todo momento se ha tratado la clase Tumor (en los modelos la clase 1 o 2, en alguna de ellas) como la clase positiva.

En el caso de los modelos *Elastic Net*, *Random Forest* y *SVM* se han realizado los modelos tanto ponderando como no, con tal de observar si estos son suficientemente robustos sin la ponderación y ver cómo esta afecta a la predicción.

Para comparar entre modelos, se ha observado, además de las métricas obtenidas mediante la función creada antes, la matriz de confusión junto con sus estadísticos de forma individual.

```

# Elastic Net (ponderado).
set.seed(123)
# Se hace la validación cruzada con 'cv.glmnet()', a su vez realizando un
# balance entre clases.
cvfit <- cv.glmnet(samples_train, train_type, family = "binomial", alpha = 0.5,

```

```

        type.measure = "class", nfolds = 5,
        weights = ifelse(train_type == 1, 2, 1))

# Se obtiene el valor de lambda óptimo según la validación cruzada.
best_lambda <- cvfit$lambda.min

# Y con este se ajusta el modelo Elastic Net, usando la ahora 'glmnet()'.
elastic_net_pond <- glmnet(samples_train, train_type, family = "binomial",
                           alpha = 0.5, lambda = best_lambda)

# Se realizan las predicciones sobre el conjunto de validación.
y_elastic_net <- factor(ifelse(predict(elastic_net_pond, samples_val,
                                     type = "response") > 0.5, 1, 0))

# Y se genera y muestra la matriz de confusión de estas predicciones. También se
# calculan las métricas con la función generada previamente.
cm_en_pond <-
  confusionMatrix(validation_type, y_elastic_net, positive = "1")
en_pond_metrics <- metrics_samples(cm_en_pond, "EN (pond.)")
cm_en_pond

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  42    2
##           1   2 323
##
##           Accuracy : 0.9892
##           95% CI : (0.9725, 0.997)
##       No Information Rate : 0.8808
##       P-Value [Acc > NIR] : 1.245e-15
##
##           Kappa : 0.9484
##
##  Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9938
##           Specificity : 0.9545
##       Pos Pred Value : 0.9938
##       Neg Pred Value : 0.9545
##           Prevalence : 0.8808
##       Detection Rate : 0.8753
##   Detection Prevalence : 0.8808
##       Balanced Accuracy : 0.9742
##
##           'Positive' Class : 1
##

```

```

# Elastic Net (sin ponderar).

# se realiza el mismo proceso que antes, pero esta vez sin realizar el balance
# entre clases, es decir, la ponderación.
set.seed(123)
cvfit <- cv.glmnet(samples_train, train_type, family = "binomial", alpha = 0.5,
                  type.measure = "class", nfolds = 5)

```

```

best_lambda <- cvfit$lambda.min

elastic_net <- glmnet(samples_train, train_type, family = "binomial",
                      alpha = 0.5, lambda = best_lambda)

y_elastic_net <- factor(ifelse(predict(elastic_net, samples_val,
                                     type = "response") > 0.5, 1, 0))

cm_en <-
  confusionMatrix(validation_type, y_elastic_net, positive = "1")
en_metrics <- metrics_samples(cm_en, "EN")
cm_en

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##              0  42    2
##              1   3 322
##
##              Accuracy : 0.9864
##              95% CI : (0.9687, 0.9956)
##      No Information Rate : 0.878
##      P-Value [Acc > NIR] : 4.57e-15
##
##              Kappa : 0.9361
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9938
##              Specificity : 0.9333
##              Pos Pred Value : 0.9908
##              Neg Pred Value : 0.9545
##              Prevalence : 0.8780
##              Detection Rate : 0.8726
##              Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9636
##
##              'Positive' Class : 1
##

```

```

# Random Forest (ponderado).
set.seed(123)
# Como Random Forest necesita que los clasificadores se encuentren en formato
# de factor y se tienen en formato numérico, se genera un nuevo objeto con
# los datos transformado con 'as.factor()'.
train_type_f <- as.factor(train_type)

# Se genera un vector con tres posibles valores para 'mtry'.
mtry_values <- floor(c(sqrt(ncol(train_samples)),
                      ncol(train_samples)/3, ncol(train_samples)/2))

# Y también se crean los folds para la validación cruzada con 'createFolds()'.
folds <- createFolds(train_type_f, k = 5, list = TRUE, returnTrain = FALSE)
# Para poder guardar los resultados de las validaciones cruzadas, se crea un
# vector numérico de la misma longitud que el vector 'mtry_values'.

```

```

cv_results <- numeric(length(mtry_values))

for(mtry in seq_along(mtry_values)) {
  # Se genera un vector para almacenar la precisión de cada fold.
  fold_accuracies <- numeric(length(folds))
  # Y se realiza la validación cruzada.
  for (fold in seq_along(folds)) {
    validation_idx <- folds[[fold]]
    train_idx <- setdiff(seq_len(nrow(train_samples)), validation_idx)

    # Se entrena el modelo usando 'randomForest()' con el balanceo entre clases.
    rf_fold <-
      randomForest(x = train_samples[train_idx, ],
                   y = train_type_f[train_idx],
                   mtry = mtry_values[mtry], ntree = 500,
                   classwt = c("1" = 1 / sum(train_type_f[train_idx] == "1"),
                                "2" = 1 / sum(train_type_f[train_idx] == "2")))

    # Se obtiene y guardan las predicciones del fold.
    y_pred_fold <- predict(rf_fold, newdata = train_samples[validation_idx, ])
    # Y también la precisión de este.
    fold_accuracies[fold] <- mean(y_pred_fold == train_type_f[validation_idx])
  }

  # Se guarda en 'cv_results' la media de las precisiones obtenidas para cada
  # valor de 'mtry'.
  cv_results[mtry] <- mean(fold_accuracies)
}

# Se obtiene el mejor valor de 'mtry'.
best_mtry <- mtry_values[which.max(cv_results)]

# Y se genera de nuevo el Random Forest, esta vez con el mejor valor de 'mtry'.
random_forest_pond <-
  randomForest(x = train_samples, y = train_type_f, mtry = best_mtry,
               ntree = 500, classwt = c("1" = 1 / sum(train_type_f == "1"),
                                         "2" = 1 / sum(train_type_f == "2")),
               importance = TRUE)

# Se obtienen las predicciones del conjunto de validación
y_rf <- predict(random_forest_pond, newdata = samples_val)

# Para poder generar la matriz de confusión, es necesario ajustar los niveles
# de nuevo a los valores '1' y '2'.
val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
                     levels = c("1", "2"))

# Se genera y muestra la matriz de confusión, a la vez que se obtienen las
# métricas del modelo.
cm_rf_pond <- confusionMatrix(val_type_f, y_rf, positive = "2")
rf_pond_metrics <- metrics_samples(cm_rf_pond, "RF (pond.)")
cm_rf_pond

```

```

## Confusion Matrix and Statistics
##
##           Reference

```

```
## Prediction    1    2
##              1  40   4
##              2   1 324
##
##              Accuracy : 0.9864
##              95% CI : (0.9687, 0.9956)
##              No Information Rate : 0.8889
##              P-Value [Acc > NIR] : 2.527e-13
##
##              Kappa : 0.9335
##
## Mcnemar's Test P-Value : 0.3711
##
##              Sensitivity : 0.9878
##              Specificity : 0.9756
##              Pos Pred Value : 0.9969
##              Neg Pred Value : 0.9091
##              Prevalence : 0.8889
##              Detection Rate : 0.8780
##              Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9817
##
##              'Positive' Class : 2
##
```

```
# Random Forest (sin ponderar).
```

```
# De nuevo, se genera el modelo Random Forest, pero esta vez sin añadir la  
# ponderación entre clases.
```

```
set.seed(123)
mtry_values <- floor(c(sqrt(ncol(train_samples)),
                        ncol(train_samples)/3, ncol(train_samples)/2))

folds <- createFolds(train_type_f, k = 5, list = TRUE, returnTrain = FALSE)
cv_results <- numeric(length(mtry_values))

for(mtry in seq_along(mtry_values)) {
  fold_accuracies <- numeric(length(folds))

  for (fold in seq_along(folds)) {
    validation_idx <- folds[[fold]]
    train_idx <- setdiff(seq_len(nrow(train_samples)), validation_idx)

    rf_fold <-
      randomForest(x = train_samples[train_idx, ],
                   y = train_type_f[train_idx],
                   mtry = mtry_values[mtry], ntree = 500)

    y_pred_fold <- predict(rf_fold, newdata = train_samples[validation_idx, ])

    fold_accuracies[fold] <- mean(y_pred_fold == train_type_f[validation_idx])
  }

  cv_results[mtry] <- mean(fold_accuracies)
}

best_mtry <- mtry_values[which.max(cv_results)]
```

```

random_forest <-
  randomForest(x = train_samples, y = train_type_f, mtry = best_mtry,
               ntree = 500, importance = TRUE)

y_rf <- predict(random_forest, newdata = samples_val)

val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
                     levels = c("1", "2"))

cm_rf <- confusionMatrix(val_type_f, y_rf, positive = "2")
rf_metrics <- metrics_samples(cm_rf, "RF")
cm_rf

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    1    2
##           1  42    2
##           2   1 324
##
##              Accuracy : 0.9919
##              95% CI : (0.9764, 0.9983)
##      No Information Rate : 0.8835
##      P-Value [Acc > NIR] : 2.831e-16
##
##              Kappa : 0.9609
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9939
##              Specificity : 0.9767
##              Pos Pred Value : 0.9969
##              Neg Pred Value : 0.9545
##              Prevalence : 0.8835
##              Detection Rate : 0.8780
##      Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9853
##
##              'Positive' Class : 2
##

```

```

# SVM lineal (ponderado).
set.seed(123)
# Se genera el modelo usando la función 'svm()'. Se indica mediante que se use
# un modelo lineal con "kernel = 'linear'". Se añade el balanceo entre clases y
# también la validación cruzada con 'tuneControl()'.
svm_model_pond <- svm(x = train_samples, y = train_type_f,
                     kernel = "linear", class.weights = c("1" = 2, "2" = 1),
                     tunecontrol = tune.control(cross = 5))

# Se predicen las muestras del conjunto de validación.
y_svm <- predict(svm_model_pond, newdata = samples_val)

# Y se ajustan los niveles para generar la matriz de confusión.
val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
                     levels = c("1", "2"))

```

```

# Se crea e imprime la matriz de confusión, además de que se obtienen los
# estadísticos del modelo.
cm_svm_pond <- confusionMatrix(val_type_f, y_svm, positive = "2")
svm_pond_metrics <- metrics_samples(cm_svm_pond, "SVM (pond.)")
cm_svm_pond

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2
##           1  42   2
##           2   2 323
##
##           Accuracy : 0.9892
##           95% CI : (0.9725, 0.997)
##       No Information Rate : 0.8808
##       P-Value [Acc > NIR] : 1.245e-15
##
##           Kappa : 0.9484
##
##  Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9938
##           Specificity : 0.9545
##       Pos Pred Value : 0.9938
##       Neg Pred Value : 0.9545
##           Prevalence : 0.8808
##       Detection Rate : 0.8753
##   Detection Prevalence : 0.8808
##       Balanced Accuracy : 0.9742
##
##       'Positive' Class : 2
##

```

```

# SVM lineal (sin ponderar).

```

```

# Se realiza exactamente el mismo código, pero eliminando el balanceo entre
# clases en la generación del modelo.

```

```

set.seed(123)
svm_model <- svm(x = train_samples, y = train_type_f, kernel = "linear",
  tunecontrol = tune.control(cross = 5))

y_svm <- predict(svm_model, newdata = samples_val)

val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
  levels = c("1", "2"))

cm_svm <- confusionMatrix(val_type_f, y_svm, positive = "2")
svm_metrics <- metrics_samples(cm_svm, "SVM")
cm_svm

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2

```



```
##          1  42   2
##          2   2 323
##
##          Accuracy : 0.9892
##          95% CI : (0.9725, 0.997)
##      No Information Rate : 0.8808
##      P-Value [Acc > NIR] : 1.245e-15
##
##          Kappa : 0.9484
##
##      McNemar's Test P-Value : 1
##
##          Sensitivity : 0.9938
##          Specificity : 0.9545
##      Pos Pred Value : 0.9938
##      Neg Pred Value : 0.9545
##          Prevalence : 0.8808
##      Detection Rate : 0.8753
##      Detection Prevalence : 0.8808
##      Balanced Accuracy : 0.9742
##
##      'Positive' Class : 2
##
```

```
# XGBoost.
set.seed(123)
# Se prepara una matriz de entrenamiento mediante 'xgb.DMatrix()'. En este caso,
# las etiquetas se encuentran junto a la matriz, y se convierten de nuevo a
# valores de '0' y '1'.
dmatrix <-
  xgb.DMatrix(data = as.matrix(train_samples),
              label = as.numeric(as.character(train_cancer_type$SampleType)) - 1)

# Se definen los parámetros. Se indica que se quiere una clasificación binaria y
# al balanceo entre clases.
params <- list(
  objective = "binary:logistic",
  scale_pos_weight = sum(train_cancer_type$SampleType == 2) /
    sum(train_cancer_type$SampleType == 1))

# Se realiza la validación cruzada con 'xgb.cv()', indicando las rondas, los
# folds y otros parámetros de importancia.
cv_xgboost <- xgb.cv(params = params, data = dmatrix, nrounds = 100, nfold = 5,
                    stratified = TRUE, early_stopping_rounds = 10, verbose = 0)

# Con todo preparado, se entrena el modelo con el número óptimo de iteraciones
# obtenido mediante la validación cruzada.
xgb_model <- xgb.train(params = params, data = dmatrix,
                      nrounds = cv_xgboost$best_iteration, verbose = 0)

# Se predicen las clases del conjunto de validación.
y_xgb <- factor(ifelse(predict(xgb_model, newdata= samples_val) > 0.5, 1, 0))

# Se genera la matriz de confusión y se obtienen las métricas.
cm_xgb <- confusionMatrix(validation_type, y_xgb, positive = "1")
xgb_metrics <- metrics_samples(cm_xgb, "XGBoost")
# Se imprime la matriz de confusión.
```

cm_xgb

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  42    2
##           1    1 324
##
##           Accuracy : 0.9919
##           95% CI : (0.9764, 0.9983)
##           No Information Rate : 0.8835
##           P-Value [Acc > NIR] : 2.831e-16
##
##           Kappa : 0.9609
##
## Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9939
##           Specificity : 0.9767
##           Pos Pred Value : 0.9969
##           Neg Pred Value : 0.9545
##           Prevalence : 0.8835
##           Detection Rate : 0.8780
##           Detection Prevalence : 0.8808
##           Balanced Accuracy : 0.9853
##
##           'Positive' Class : 1
##
```

```
# Red neuronal con clases ponderadas.
set.seed(123)
# En el caso de la red neuronal, es necesario que los datos se encuentren forma
# de matriz y que los clasificadores (en clasificación binaria) tengan las
# etiquetas de '0' y '1'.
train_matrix <- as.matrix(train_samples)
train_type_cnn <- ifelse(train_cancer_type$SampleType == 2, 1, 0)

# Se prepara el balanceo entre clases.
class_weights <-
  list("0" = length(train_type_cnn) / (2 * sum(train_type_cnn == 0)),
       "1" = length(train_type_cnn) / (2 * sum(train_type_cnn == 1)))

# Se definen las capas del modelo.
cnn_model <- keras_model_sequential() %>%
  layer_dense(units = 64, activation = "relu",
              input_shape = ncol(train_matrix)) %>%
  layer_dropout(0.3) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dense(units = 1, activation = "sigmoid")

# Y se realiza la compilación.
cnn_model %>% compile(loss = "binary_crossentropy",
                    optimizer = optimizer_adam(),
                    metrics = c("accuracy"))
```

```

# Con la compilación realizada, se entrena el modelo con validación interna y
# el balanceo de clases.
history_cnn <- cnn_model %>% fit(train_matrix, train_type_cnn, epochs = 30,
                                batch_size = 32, class_weight = class_weights,
                                validation_split = 0.2, verbose = 0)

# Se llevan a cabo las predicciones sobre el conjunto de validación.
y_cnn <- factor(ifelse(predict(cnn_model, samples_val) >= 0.5, 1, 0),
                levels = c("0", "1"))

```

```
## 12/12 - 0s - 8ms/step
```

```

# Y se genera su matriz de confusión. De esta, se obtienen los estadísticos.
cm_cnn <- confusionMatrix(validation_type, y_cnn, positive = "1")
cnn_metrics <- metrics_samples(cm_cnn, "CNN")
# Se muestra el resultado de la matriz de confusión.
cm_cnn

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0    0  44
##           1    0 325
##
##           Accuracy : 0.8808
##           95% CI : (0.8432, 0.912)
##    No Information Rate : 1
##    P-Value [Acc > NIR] : 1
##
##           Kappa : 0
##
## Mcnemar's Test P-Value : 9.022e-11
##
##           Sensitivity : 0.8808
##           Specificity :      NA
##    Pos Pred Value :      NA
##    Neg Pred Value :      NA
##           Prevalence : 1.0000
##    Detection Rate : 0.8808
##    Detection Prevalence : 0.8808
##    Balanced Accuracy :      NA
##
##    'Positive' Class : 1
##

```

Con los modelos creados y habiendo observado los resultados de cada matriz de confusión, también se ha realizado una tabla que contiene los estadísticos más importantes a la hora de poder escoger qué modelo es el que funciona mejor.

```

# Se juntan todos los objetos que contienen los estadísticos de los modelos con
# 'rbind()' y se almacenan en un nuevo objeto.
metrics_models <-
  rbind(en_pond_metrics, en_metrics, rf_pond_metrics, rf_metrics,
        svm_pond_metrics, svm_metrics, xgb_metrics, cnn_metrics)

```

```
# Se muestran estos resultados usando 'kable()'.
kable(metrics_models,
      caption = "Métricas de los modelos para predecir el tipo de muestra") %>%
  kable_styling()
```

Table 6: Métricas de los modelos para predecir el tipo de muestra

Model	Prec_n	Prec_t	Precision	Recall_n	Recall	F1_n	F1_macro	Kappa	Spec	Bal_acc
EN (pond.)	0.9545	0.9938	0.9742	0.9545	0.9742	0.9545	0.9742	0.9484	0.9545	0.9742
EN	0.9545	0.9908	0.9726	0.9333	0.9636	0.9438	0.9680	0.9361	0.9333	0.9636
RF (pond.)	0.9091	0.9969	0.9530	0.9756	0.9817	0.9412	0.9668	0.9335	0.9756	0.9817
RF	0.9545	0.9969	0.9757	0.9767	0.9853	0.9655	0.9804	0.9609	0.9767	0.9853
SVM (pond.)	0.9545	0.9938	0.9742	0.9545	0.9742	0.9545	0.9742	0.9484	0.9545	0.9742
SVM	0.9545	0.9938	0.9742	0.9545	0.9742	0.9545	0.9742	0.9484	0.9545	0.9742
XGBoost	0.9545	0.9969	0.9757	0.9767	0.9853	0.9655	0.9804	0.9609	0.9767	0.9853
CNN	0.0000	1.0000	0.5000	NaN	NaN	NaN	NaN	0.0000	NA	NA

Se ha observado que la mayoría de los modelos han resultado en unos estadísticos muy elevados, excepto el caso de la red neuronal convolucional (CNN), donde el modelo se ha sesgado en una sola clase y no ha sido capaz de clasificar ninguna de las muestras como muestra normal. Esto se debe principalmente al desequilibrio de clases que hay en los datos (hay aproximadamente un total de 1500 muestras tumorales más que normales).

Por ello, se ha realizado un submuestro de las muestras tumorales de forma aleatoria, de manera que en el conjunto de entrenamiento este formado por un 25% de muestras normales y un 75% de muestras tumorales.

```
set.seed(123)
# Se crea una función para realizar el submuestreo.
undersample_indices <- function(y, minority_label = 0, ratio_minor = 0.25) {
  # Se obtiene los índices de la clase minoritaria y la mayoritaria.
  idx_minor <- which(y == minority_label)
  idx_major <- which(y != minority_label)

  # Número objetivo de muestras de la clase mayoritaria según el ratio deseado.
  target_major <- round(length(idx_minor) * ((1 - ratio_minor) / ratio_minor))
  # Se realiza una selección aleatoria de la clase mayoritaria.
  idx_major_sub <- sample(idx_major, min(target_major, length(idx_major)))
  # Y se devuelven los índices de forma combinada y ordenada.
  sort(c(idx_minor, idx_major_sub))
}

# Índices finales tras el submuestreo.
idx_final <- undersample_indices(train_type_cnn, minority_label = 0,
                                ratio_minor = 0.25)

# Se almacenan en dos nuevos objetos los conjuntos de datos balanceados.
samples_train_bal <- samples_train[idx_final, ]
train_type_bal <- train_type[idx_final]
```

Con estos nuevos datos filtrados para que la proporción se encuentre algo más equilibrada, se ha procedido a la generación de nuevo de los modelos, usando otra vez (o no) la ponderación. De esta forma, se puede estudiar cómo el equilibrio de las muestras afecta a la clasificación de las clases, y si la reducción de la clase mayoritaria conlleva que se dé una mejor clasificación de estos.

```

# Elastic Net (ponderado).
set.seed(123)
# Se hace la validación cruzada con 'cv.glmnet()', a su vez realizando un
# balance entre clases.
cvfit <- cv.glmnet(samples_train_bal, train_type_bal, family = "binomial",
                  alpha = 0.5, type.measure = "class", nfolds = 5,
                  weights = ifelse(train_type_bal == 1, 2, 1))

# Se obtiene el valor de lambda óptimo según la validación cruzada.
best_lambda <- cvfit$lambda.min

# Y con este se ajusta el modelo Elastic Net, usando la ahora 'glmnet()'.
elastic_net_pond <- glmnet(samples_train_bal, train_type_bal,
                          family = "binomial", alpha = 0.5,
                          lambda = best_lambda)

# Se realizan las predicciones sobre el conjunto de validación.
y_elastic_net <- factor(ifelse(predict(elastic_net_pond, samples_val,
                                     type = "response") > 0.5, 1, 0))

# Y se genera y muestra la matriz de confusión de estas predicciones. También se
# calculan las métricas con la función generada previamente.
cm_en_pond2 <-
  confusionMatrix(validation_type, y_elastic_net, positive = "1")
en_pond_metrics2 <- metrics_samples(cm_en_pond2, "EN (pond.)")
cm_en_pond2

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##              0  42    2
##              1   3 322
##
##              Accuracy : 0.9864
##              95% CI : (0.9687, 0.9956)
##              No Information Rate : 0.878
##              P-Value [Acc > NIR] : 4.57e-15
##
##              Kappa : 0.9361
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9938
##              Specificity : 0.9333
##              Pos Pred Value : 0.9908
##              Neg Pred Value : 0.9545
##              Prevalence : 0.8780
##              Detection Rate : 0.8726
##              Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9636
##
##              'Positive' Class : 1
##

```

```

# Elastic Net (sin ponderar).

# Se realiza el mismo proceso que antes, pero esta vez sin realizar el balance
# entre clases, es decir, la ponderación.
set.seed(123)
cvfit <- cv.glmnet(samples_train_bal, train_type_bal, family = "binomial",
                  alpha = 0.5, type.measure = "class", nfolds = 5)

best_lambda <- cvfit$lambda.min

elastic_net <- glmnet(samples_train_bal, train_type_bal, family = "binomial",
                    alpha = 0.5, lambda = best_lambda)

y_elastic_net <- factor(ifelse(predict(elastic_net, samples_val,
                                     type = "response") > 0.5, 1, 0))

cm_en2 <-
  confusionMatrix(validation_type, y_elastic_net, positive = "1")
en_metrics2 <- metrics_samples(cm_en2, "EN")
cm_en2

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##           0 42    2
##           1   3 322
##
##              Accuracy : 0.9864
##              95% CI : (0.9687, 0.9956)
##      No Information Rate : 0.878
##      P-Value [Acc > NIR] : 4.57e-15
##
##              Kappa : 0.9361
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9938
##              Specificity : 0.9333
##              Pos Pred Value : 0.9908
##              Neg Pred Value : 0.9545
##              Prevalence : 0.8780
##              Detection Rate : 0.8726
##              Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9636
##
##              'Positive' Class : 1
##

```

```

# Random Forest (ponderado).
set.seed(123)
# Como Random Forest necesita que los clasificadores se encuentren en formato
# de factor y se tienen en formato numérico, se genera un nuevo objeto con
# los datos transformado con 'as.factor()'.
train_type_f <- as.factor(train_type_bal)

```

```

# Se genera un vector con tres posibles valores para 'mtry'.
mtry_values <- floor(c(sqrt(ncol(samples_train_bal)),
                        ncol(samples_train_bal)/3, ncol(samples_train_bal)/2))

# Y también se crean los folds para la validación cruzada con 'createFolds()'.
folds <- createFolds(train_type_f, k = 5, list = TRUE, returnTrain = FALSE)
# Para poder guardar los resultados de las validaciones cruzadas, se crea un
# vector numérico de la misma longitud que el vector 'mtry_values'.
cv_results <- numeric(length(mtry_values))

for(mtry in seq_along(mtry_values)) {
  # Se genera un vector para almacenar la precisión de cada fold.
  fold_accuracies <- numeric(length(folds))
  # Y se realiza la validación cruzada.
  for (fold in seq_along(folds)) {
    validation_idx <- folds[[fold]]
    train_idx <- setdiff(seq_len(nrow(samples_train_bal)), validation_idx)

    # Se entrena el modelo usando 'randomForest()' con el balanceo entre clases.
    rf_fold <-
      randomForest(x = samples_train_bal[train_idx, ],
                  y = train_type_f[train_idx],
                  mtry = mtry_values[mtry], ntree = 500,
                  classwt = c("1" = 1 / sum(train_type_f[train_idx] == "1"),
                              "2" = 1 / sum(train_type_f[train_idx] == "2"))))

    # Se obtiene y guardan las predicciones del fold.
    y_pred_fold <- predict(rf_fold,
                          newdata = samples_train_bal[validation_idx, ])
    # Y también la precisión de este.
    fold_accuracies[fold] <- mean(y_pred_fold == train_type_f[validation_idx])
  }
  # Se guarda en 'cv_results' la media de las precisiones obtenidas para cada
  # valor de 'mtry'.
  cv_results[mtry] <- mean(fold_accuracies)
}

# Se obtiene el mejor valor de 'mtry'.
best_mtry <- mtry_values[which.max(cv_results)]

# Y se genera de nuevo el Random Forest, esta vez con el mejor valor de 'mtry'.
random_forest_pond <-
  randomForest(x = samples_train_bal, y = train_type_f, mtry = best_mtry,
              ntree = 500, classwt = c("1" = 1 / sum(train_type_f == "1"),
                                      "2" = 1 / sum(train_type_f == "2")),
              importance = TRUE)

# Se obtienen las predicciones del conjunto de validación
y_rf <- predict(random_forest_pond, newdata = samples_val)

# Para poder generar la matriz de confusión, es necesario ajustar los niveles
# de nuevo a los valores '1' y '2'.
val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
                    levels = c("1", "2"))

# Se genera y muestra la matriz de confusión, a la vez que se obtienen las
# métricas del modelo.

```

```
cm_rf_pond2 <- confusionMatrix(val_type_f, y_rf, positive = "2")
rf_pond_metrics2 <- metrics_samples(cm_rf_pond2, "RF (pond.)")
cm_rf_pond2
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1    2
##           1  41   3
##           2   3 322
##
##           Accuracy : 0.9837
##           95% CI : (0.9649, 0.994)
##           No Information Rate : 0.8808
##           P-Value [Acc > NIR] : 1.056e-13
##
##           Kappa : 0.9226
##
##           Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9908
##           Specificity : 0.9318
##           Pos Pred Value : 0.9908
##           Neg Pred Value : 0.9318
##           Prevalence : 0.8808
##           Detection Rate : 0.8726
##           Detection Prevalence : 0.8808
##           Balanced Accuracy : 0.9613
##
##           'Positive' Class : 2
##
```

```
# Random Forest (sin ponderar).
```

```
# De nuevo, se genera el modelo Random Forest, pero esta vez sin añadir la
# ponderación entre clases.
set.seed(123)
train_type_f <- as.factor(train_type_bal)
mtry_values <- floor(c(sqrt(ncol(samples_train_bal)),
                        ncol(samples_train_bal)/3, ncol(samples_train_bal)/2))

folds <- createFolds(train_type_f, k = 5, list = TRUE, returnTrain = FALSE)
cv_results <- numeric(length(mtry_values))

for(mtry in seq_along(mtry_values)) {
  fold_accuracies <- numeric(length(folds))

  for (fold in seq_along(folds)) {
    validation_idx <- folds[[fold]]
    train_idx <- setdiff(seq_len(nrow(samples_train_bal)), validation_idx)

    rf_fold <-
      randomForest(x = samples_train_bal[train_idx, ],
                   y = train_type_f[train_idx],
                   mtry = mtry_values[mtry], ntree = 500)
```



```

y_pred_fold <- predict(rf_fold,
                      newdata = samples_train_bal[validation_idx, ])

fold_accuracies[fold] <- mean(y_pred_fold == train_type_f[validation_idx])
}

cv_results[mtry] <- mean(fold_accuracies)
}

best_mtry <- mtry_values[which.max(cv_results)]

random_forest <-
  randomForest(x = samples_train_bal, y = train_type_f, mtry = best_mtry,
              ntree = 500, importance = TRUE)

y_rf <- predict(random_forest, newdata = samples_val)

val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
                    levels = c("1", "2"))

cm_rf2 <- confusionMatrix(val_type_f, y_rf, positive = "2")
rf_metrics2 <- metrics_samples(cm_rf2, "RF")
cm_rf2

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    1    2
##              1  42    2
##              2   1 324
##
##              Accuracy : 0.9919
##              95% CI : (0.9764, 0.9983)
##              No Information Rate : 0.8835
##              P-Value [Acc > NIR] : 2.831e-16
##
##              Kappa : 0.9609
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9939
##              Specificity : 0.9767
##              Pos Pred Value : 0.9969
##              Neg Pred Value : 0.9545
##              Prevalence : 0.8835
##              Detection Rate : 0.8780
##              Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9853
##
##              'Positive' Class : 2
##

```

```

# SVM lineal (ponderado).
set.seed(123)
# Se genera el modelo usando la función 'svm()'. Se indica mediante que se use
# un modelo lineal con "kernel = 'linear'". Se añade el balanceo entre clases y

```

```

# también la validación cruzada con 'tuneControl()'.
svm_model_pond <- svm(x = samples_train_bal, y = train_type_f,
                     kernel = "linear", class.weights = c("1" = 2, "2" = 1),
                     tunecontrol = tune.control(cross = 5))

# Se predicen las muestras del conjunto de validación.
y_svm <- predict(svm_model_pond, newdata = samples_val)

# Y se ajustan los niveles para generar la matriz de confusión.
val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
                     levels = c("1", "2"))

# Se crea e imprime la matriz de confusión, además de que se obtienen los
# estadísticos del modelo.
cm_svm_pond2 <- confusionMatrix(val_type_f, y_svm, positive = "2")
svm_pond_metrics2 <- metrics_samples(cm_svm_pond2, "SVM (pond.)")
cm_svm_pond2

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    1    2
##           1  42    2
##           2   3 322
##
##              Accuracy : 0.9864
##              95% CI : (0.9687, 0.9956)
##      No Information Rate : 0.878
##      P-Value [Acc > NIR] : 4.57e-15
##
##              Kappa : 0.9361
##
##  Mcnemar's Test P-Value : 1
##
##      Sensitivity : 0.9938
##      Specificity : 0.9333
##      Pos Pred Value : 0.9908
##      Neg Pred Value : 0.9545
##      Prevalence : 0.8780
##      Detection Rate : 0.8726
##      Detection Prevalence : 0.8808
##      Balanced Accuracy : 0.9636
##
##      'Positive' Class : 2
##

```

```

# SVM lineal (sin ponderar).

# Se realiza exactamente el mismo código, pero eliminando el balanceo entre
# clases en la generación del modelo.
set.seed(123)
svm_model <- svm(x = samples_train_bal, y = train_type_f, kernel = "linear",
                 tunecontrol = tune.control(cross = 5))

y_svm <- predict(svm_model, newdata = samples_val)

```

```
val_type_f <- factor(ifelse(validation_type == 0, "1", "2"),
                      levels = c("1", "2"))

cm_svm2 <- confusionMatrix(val_type_f, y_svm, positive = "2")
svm_metrics2 <- metrics_samples(cm_svm2, "SVM")
cm_svm2
```

```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction    1    2
##           1  42   2
##           2   3 322
##
##              Accuracy : 0.9864
##              95% CI : (0.9687, 0.9956)
##      No Information Rate : 0.878
##      P-Value [Acc > NIR] : 4.57e-15
##
##              Kappa : 0.9361
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9938
##              Specificity : 0.9333
##              Pos Pred Value : 0.9908
##              Neg Pred Value : 0.9545
##              Prevalence : 0.8780
##              Detection Rate : 0.8726
##      Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9636
##
##              'Positive' Class : 2
##
```

```
# XGBoost.
set.seed(123)
# Se prepara una matriz de entrenamiento mediante 'xgb.DMatrix()'. En este caso,
# las etiquetas se encuentran junto a la matriz, y se convierten de nuevo a
# valores de '0' y '1'.
dmatrix <-
  xgb.DMatrix(data = as.matrix(samples_train_bal),
              label = as.numeric(as.character(train_type_bal)) - 1)

# Se definen los parámetros. Se indica que se quiere una clasificación binaria y
# al balanceo entre clases.
params <- list(
  objective = "binary:logistic",
  scale_pos_weight = sum(train_type_bal == 2) /
    sum(train_type_bal == 1))

# Se realiza la validación cruzada con 'xgb.cv()', indicando las rondas, los
# folds y otros parámetros de importancia.
cv_xgboost <- xgb.cv(params = params, data = dmatrix, nrounds = 100, nfold = 5,
                     stratified = TRUE, early_stopping_rounds = 10, verbose = 0)
```

```

# Con todo preparado, se entrena el modelo con el número óptimo de iteraciones
# obtenido mediante la validación cruzada.
xgb_model <- xgb.train(params = params, data = dmatrix,
                      nrounds = cv_xgboost$best_iteration, verbose = 0)

# Se predicen las clases del conjunto de validación.
y_xgb <- factor(ifelse(predict(xgb_model, newdata= samples_val) > 0.5, 1, 0))

# Se genera la matriz de confusión y se obtienen las métricas.
cm_xgb2 <- confusionMatrix(validation_type, y_xgb, positive = "1")
xgb_metrics2 <- metrics_samples(cm_xgb2, "XGBoost")
# Se imprime la matriz de confusión.
cm_xgb2

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##              0  42    2
##              1   2 323
##
##              Accuracy : 0.9892
##              95% CI : (0.9725, 0.997)
##              No Information Rate : 0.8808
##              P-Value [Acc > NIR] : 1.245e-15
##
##              Kappa : 0.9484
##
##  Mcnemar's Test P-Value : 1
##
##              Sensitivity : 0.9938
##              Specificity : 0.9545
##              Pos Pred Value : 0.9938
##              Neg Pred Value : 0.9545
##              Prevalence : 0.8808
##              Detection Rate : 0.8753
##              Detection Prevalence : 0.8808
##              Balanced Accuracy : 0.9742
##
##              'Positive' Class : 1
##

```

```

set.seed(123)
# En el caso de la red neuronal, es necesario que los datos se encuentren forma
# de matriz y que los clasificadores (en clasificación binaria) tengan las
# etiquetas de '0' y '1'.
train_matrix <- as.matrix(samples_train_bal)
train_type_cnn <- ifelse(train_type_bal == 2, 1, 0)

# Se prepara el balanceo entre clases.
class_weights <-
  list("0" = length(train_type_cnn) / (2 * sum(train_type_cnn == 0)),
       "1" = length(train_type_cnn) / (2 * sum(train_type_cnn == 1)))

# Se definen las capas del modelo.
cnn_model <- keras_model_sequential() %>%

```

```

layer_dense(units = 64, activation = "relu",
             input_shape = ncol(train_matrix)) %>%
layer_dropout(0.3) %>%
layer_dense(units = 32, activation = "relu") %>%
layer_dense(units = 1, activation = "sigmoid")

# Y se realiza la compilación.
cnn_model %>% compile(loss = "binary_crossentropy",
                     optimizer = optimizer_adam(),
                     metrics = c("accuracy"))

# Con la compilación realizada, se entrena el modelo con validación interna y
# el balanceo de clases.
history_cnn <- cnn_model %>% fit(train_matrix, train_type_cnn, epochs = 30,
                                batch_size = 32, class_weight = class_weights,
                                validation_split = 0.2, verbose = 0)

# Se llevan a cabo las predicciones sobre el conjunto de validación.
y_cnn <- factor(ifelse(predict(cnn_model, samples_val) >= 0.5, 1, 0),
                levels = c("0", "1"))

```

```
## 12/12 - 0s - 8ms/step
```

```

# Y se genera su matriz de confusión. De esta, se obtienen los estadísticos.
cm_cnn2 <- confusionMatrix(validation_type, y_cnn, positive = "1")
cnn_metrics2 <- metrics_samples(cm_cnn2, "CNN")
# Se muestra el resultado de la matriz de confusión.
cm_cnn2

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  42    2
##           1   3 322
##
##           Accuracy : 0.9864
##           95% CI : (0.9687, 0.9956)
##       No Information Rate : 0.878
##       P-Value [Acc > NIR] : 4.57e-15
##
##           Kappa : 0.9361
##
##  Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9938
##           Specificity : 0.9333
##           Pos Pred Value : 0.9908
##           Neg Pred Value : 0.9545
##           Prevalence : 0.8780
##           Detection Rate : 0.8726
##       Detection Prevalence : 0.8808
##           Balanced Accuracy : 0.9636
##
##           'Positive' Class : 1
##

```

Con los modelos generados, se ha creado una tabla que contiene tanto los estadísticos obtenidos con los modelos antes del submuestreo de las muestras (con el conjunto en “bruto”) como los nuevos estadísticos. Con ello se pretende poder comparar los grupos de modelos entre sí, y de esta manera poder concluir si es necesario o no el equilibrio entre las clases para que los modelos funcionen más adecuadamente.

```
# Tabla comparativa de las métricas antes y después del submuestreo de datos.

# Se juntan todos los objetos que contienen los estadísticos de los modelos con
# 'rbind()' y se almacenan en un nuevo objeto.
metrics_models2 <-
  rbind(en_pond_metrics2, en_metrics2, rf_pond_metrics2, rf_metrics2,
        svm_pond_metrics2, svm_metrics2, xgb_metrics2, cnn_metrics2)

# Para poder realizar correctamente la comparativa, se vuelve a imprimir de
# nuevo en pantalla la tabla de los estadísticos antes del submuestreo.
kable(metrics_models2,
      caption = "Métricas antes del submuestreo") %>% kable_styling()
```

Table 7: Métricas antes del submuestreo

Model	Prec_n	Prec_t	Precision	Recall_n	Recall	F1_n	F1_macro	Kappa	Spec	Bal_acc
EN (pond.)	0.9545	0.9938	0.9742	0.9545	0.9742	0.9545	0.9742	0.9484	0.9545	0.9742
EN	0.9545	0.9908	0.9726	0.9333	0.9636	0.9438	0.9680	0.9361	0.9333	0.9636
RF (pond.)	0.9091	0.9969	0.9530	0.9756	0.9817	0.9412	0.9668	0.9335	0.9756	0.9817
RF	0.9545	0.9969	0.9757	0.9767	0.9853	0.9655	0.9804	0.9609	0.9767	0.9853
SVM (pond.)	0.9545	0.9938	0.9742	0.9545	0.9742	0.9545	0.9742	0.9484	0.9545	0.9742
SVM	0.9545	0.9938	0.9742	0.9545	0.9742	0.9545	0.9742	0.9484	0.9545	0.9742
XGBoost	0.9545	0.9969	0.9757	0.9767	0.9853	0.9655	0.9804	0.9609	0.9767	0.9853
CNN	0.0000	1.0000	0.5000	NaN	NaN	NaN	NaN	0.0000	NA	NA

```
# Y a continuación, se imprime la que contiene las métricas una vez realizado
# el submuestreo.
kable(metrics_models2,
      caption = "Métricas tras el submuestreo") %>% kable_styling()
```

Table 8: Métricas tras el submuestreo

Model	Prec_n	Prec_t	Precision	Recall_n	Recall	F1_n	F1_macro	Kappa	Spec	Bal_acc
EN (pond.)	0.9545	0.9908	0.9726	0.9333	0.9636	0.9438	0.9680	0.9361	0.9333	0.9636
EN	0.9545	0.9908	0.9726	0.9333	0.9636	0.9438	0.9680	0.9361	0.9333	0.9636
RF (pond.)	0.9318	0.9908	0.9613	0.9318	0.9613	0.9318	0.9613	0.9226	0.9318	0.9613
RF	0.9545	0.9969	0.9757	0.9767	0.9853	0.9655	0.9804	0.9609	0.9767	0.9853
SVM (pond.)	0.9545	0.9908	0.9726	0.9333	0.9636	0.9438	0.9680	0.9361	0.9333	0.9636
SVM	0.9545	0.9908	0.9726	0.9333	0.9636	0.9438	0.9680	0.9361	0.9333	0.9636
XGBoost	0.9545	0.9938	0.9742	0.9545	0.9742	0.9545	0.9742	0.9484	0.9545	0.9742
CNN	0.9545	0.9908	0.9726	0.9333	0.9636	0.9438	0.9680	0.9361	0.9333	0.9636

Se ha observado que el único modelo que presenta una mejora significativa es el creado mediante la red neuronal (**CNN**), que pasa de no ser capaz de clasificar ninguna muestra como normal a presentar unos estadísticos de rendimiento significativamente superiores. El modelo **Random Forest con ponderación** también muestra un pequeño aumento en sus métricas, indicando que el submuestreo ha sido útil en este caso para mejorar la clasificación.

Sin embargo, todos los demás modelos presentan unos resultados iguales o incluso algo peores. Esta disminución sobre todo se muestra en la precisión de la clase **Tumor** y en la *accuracy* balanceada. Esto se debe que, al eliminar aleatoriamente ciertas muestras clasificadas como tumorales, se ha reducido la capacidad de los modelos para identificar esta clase; por ende, no es tan preciso a la hora de clasificar las muestras de este tipo.

En consecuencia a todo lo esmentado anteriormente, se ha llegado a la conclusión de que no es necesario realizar un submuestreo de la clase mayoritaria para equilibrar la proporción de clases, ya que sin hacerlo la mayoría de modelos ya muestran unos resultados con valores muy elevados e incluso iguales o mejores que cuando se realiza el equilibrio.

En relación a los modelos, aquellos que presentan unos mejores resultados en las métricas han sido el modelo **XGBoost** y el **Random Forest** que no presenta ponderación de clases. Con los datos “crudos” ambos han presentado unos estadísticos idénticos. En el caso de los modelos entrenados sobre los datos equilibrados, XGBoost ha presentado unos resultados algo mejores que Random Forest sin ponderar (este ha mantenido los mismos valores que con los datos iniciales), aunque los resultados de este han seguido siendo algo peores que en el otro modelo.

Como el modelo final se entrenará sobre los modelos con la proporción inicial que presentan los datos y que se trata de unos modelos de clasificación binaria, se ha decidido escoger el modelo más simple entre las dos opciones posibles. Por ello, se ha decidido elegir el modelo **Random Forest no ponderado** como el mejor modelo para clasificar el tipo de muestras.

Modelos para la predicción del tipo de cáncer

Obtención de las muestras tumorales de los conjuntos

Para poder generar unos modelos que sean capaces de predecir correctamente el tipo de cáncer al que pertenece una muestra, es necesario trabajar solo sobre las muestras tumorales. Para ello, se ha creado una función que permita hacer este filtrado de forma rápida en los tres subconjuntos de datos.

```
# Función para obtener las muestras tumorales de los conjuntos de datos.
tumor_samples <- function(samples_data, classifiers_data) {
  # Se obtienen los clasificadores donde el tipo de muestra es de 2; es decir,
  # que se identifiquen como 'Tumor'.
  tumor_classifiers <- classifiers_data[classifiers_data$SampleType == 2, ]
  # Se obtienen aquellas muestras que se cuyo identificador se encuentra entre
  # los acabados de filtrar.
  tumor_samples <- samples_data[rownames(samples_data) %in%
                                tumor_classifiers$Samples, ]

  # Y se devuelve una lista en la que se incluyen por un lado los datos de las
  # muestras tumorales y por otro los clasificadores de estas.
  return(list(samples = tumor_samples, classifiers = tumor_classifiers))
}

# Se utiliza la función sobre los tres conjuntos de datos, y se obtienen los
# valores de expresión génica y los clasificadores de forma separada.
train_tumor <- tumor_samples(train_samples, train_cancer_type)
train_tumor_samples <- train_tumor$samples
train_tumor_type <- train_tumor$classifiers

test_tumor <- tumor_samples(test_samples, test_cancer_type)
test_tumor_samples <- test_tumor$samples
test_tumor_type <- test_tumor$classifiers

validation_tumor <- tumor_samples(validation_samples, validation_cancer_type)
```

```
validation_tumor_samples <- validation_tumor$samples
validation_tumor_type <- validation_tumor$classifiers
```

Generación de los modelos

De la misma forma que para los modelos anteriores, se ha generado una función capaz de obtener las métricas más importantes de la matriz de confusión. En esta caso, al tratarse de modelos predictivos multiclase y no binarios, los estadísticos no se pueden obtener de una forma tan directa de la matriz. La diferencia es que en este caso se debe calcular la media de cada uno de ellos.

La información de los estadísticos de cada modelo se almacenan en un *dataframe*, de manera que así se puede generar una tabla comparativa al haber generado todos los modelos más fácilmente.

```
# Función para obtener las métricas para el tipo de cáncer.
metrics_ct <- function(cm, model_name = "Modelo") {

  # La exactitud y el coeficiente Kappa se obtienen de manera directa de la
# matriz de confusión.
  accuracy <- round(unname(cm$overall["Accuracy"]), 4)
  kappa <- round(unname(cm$overall["Kappa"]), 4)

  # Se obtiene en formato dataframe el apartado 'byClass' de la matriz.
  byClass <- as.data.frame(cm$byClass)

  # Y de este, se obtienen el valor F1 macro, la precisión, sensibilidad,
# especificidad y 'accuracy' balanceada.
  f1_macro <- round(mean(byClass$F1, na.rm = TRUE), 4)
  precision <- round(mean(byClass$`Pos Pred Value`, na.rm = TRUE), 4)
  recall <- round(mean(byClass$Sensitivity, na.rm = TRUE), 4)
  specificity <- round(mean(byClass$Specificity, na.rm = TRUE), 4)
  balanced_accuracy <- round(mean(byClass$`Balanced Accuracy`, na.rm = TRUE), 4)

  # Se genera un dataframe con los estadísticos más importantes para realizar
# las comparaciones de modelos.
  model_metrics <-
    data.frame("Model" = model_name, "Precision" = precision,
              "Recall" = recall, "Specificity" = specificity,
              "F1_macro" = f1_macro, "Accuracy" = accuracy,
              "Kappa" = kappa, "Balanced_accuracy" = balanced_accuracy)
}
```

Junto con la función de las métricas, también se ha creado una capaz de calcular el área bajo la curva ROC (AUC-ROC, *area under the ROC curve*). Cabe tener en cuenta que, para ello, es necesario que las predicciones no solo se encuentran en forma de etiquetas, sino también como probabilidades. Los valores de AUC para cada tipo de cáncer y el valor global se almacenan en un *dataframe*.

```
# Función para obtener los valores AUC-ROC de cada tipo de cáncer y el valor
# global.
auc_roc <- function(probabilities, model_name) {

  # Se obtienen las clases del conjunto de validación
  classes <- levels(validation_type)
  # Y se genera un vector numérico con esta longitud.
  auc <- numeric(length(classes))

  # se calcula el valor AUC-ROC para cada uno respecto al resto de clases.
```



```

for (class in seq_along(classes)) {
  bin_ref <- ifelse(validation_type == classes[class], 1, 0)
  bin_pred <- probabilities[, classes[class]]
  # Se calcula ROC con 'roc()'.
  roc <- roc(bin_ref, bin_pred)
  # Y el valor AUC con 'auc()'. Se almacena este en el vector generado antes.
  auc[class] <- auc(roc)
}

# Se le asigna el nombre de cada clase a los valores de AUC obtenidos.
names(auc) <- classes

# Se genera un dataframe con los valores obtenidos para cada tipo de cáncer,
# a su vez que el valor global.
auc_values <- data.frame("Model" = model_name,
                        "BRCA" = round(unname(auc[1]), 4),
                        "KIRC" = round(unname(auc[2]), 4),
                        "LUAD" = round(unname(auc[3]), 4),
                        "LUSC" = round(unname(auc[4]), 4),
                        "PRAD" = round(unname(auc[5]), 4),
                        "THCA" = round(unname(auc[6]), 4),
                        "Global" = round(mean(auc), 4))
}

```

Los valores de expresión se han convertido a matriz y almacenado en nuevos objetos. En el caso de las clases, y como se ha hecho para los modelos predictivos para el tipo de muestra, se han almacenado en dos nuevos objetos pero manteniéndolas idénticas.

```

# Se preparan los conjuntos de datos para poder entrenar los modelos. Para ello,
# se generan dos objetos nuevos con los valores de expresión en formato de
# matriz, a su vez que otros dos que contienen solo los clasificadores del tipo
# de cáncer de cada muestra para los conjuntos de entrenamiento y de validación.
samples_train <- as.matrix(train_tumor_samples)
train_type <- train_tumor_type$CancerType
samples_val <- as.matrix(validation_tumor_samples)
validation_type <- validation_tumor_type$CancerType

# k-NN.
set.seed(123)
# Se definen diferentes valores de 'k'.
k_values <- c(1, 3, 5, 7, 9)
# Y se genera un vector con la misma longitud que valores se han añadido.
accuracy <- numeric(length(k_values))

# Para cada 'k' se entrena el modelo con 'knn.cv()' (usando validación cruzada) y
# se almacenan los resultados en el vector generado antes.
for (k in seq_along(k_values)) {
  predictions <- knn.cv(train = samples_train, cl = train_type, k = k_values[k])
  accuracy[k] <- mean(predictions == train_type)
}

# Se obtiene el mejor valor de 'k'.
best_k <- k_values[which.max(accuracy)]

# Se entrena el modelo con 'kkn()' usando el valor óptimo de 'k'.
fit_knn <- kkn(formula = train_type ~ ., train = as.data.frame(samples_train),

```

```

test = as.data.frame(samples_val), k = best_k,
kernel = "rectangular")

# Obtención de las probabilidades del entrenamiento del modelo.
knn_prob <- fit_knn$prob
# Las probabilidades obtenidas se convierten a factor para poder generar la
# matriz de confusión.
y_knn <-
  factor(apply(knn_prob, 1, function(x) colnames(knn_prob)[which.max(x)]),
         levels = levels(train_type))

# Se genera la matriz de confusión, se obtienen las métricas y los valores
# AUC-ROC y se imprime la matriz.
cm_knn <- confusionMatrix(y_knn, validation_type)
knn_metrics <- metrics_ct(cm_knn, "k-NN")
knn_auc <- auc_roc(knn_prob, "k-NN")
cm_knn

```

Confusion Matrix and Statistics

##

Reference

Prediction BRCA KIRC LUAD LUSC PRAD THCA

BRCA 51 0 0 0 0 0

KIRC 0 54 0 0 0 0

LUAD 0 0 55 5 0 0

LUSC 0 0 2 47 0 0

PRAD 0 0 0 0 55 0

THCA 0 0 0 0 0 56

##

Overall Statistics

##

Accuracy : 0.9785

95% CI : (0.9561, 0.9913)

No Information Rate : 0.1754

P-Value [Acc > NIR] : < 2.2e-16

##

Kappa : 0.9741

##

McNemar's Test P-Value : NA

##

Statistics by Class:

##

Class: BRCA Class: KIRC Class: LUAD Class: LUSC

Sensitivity 1.0000 1.0000 0.9649 0.9038

Specificity 1.0000 1.0000 0.9813 0.9927

Pos Pred Value 1.0000 1.0000 0.9167 0.9592

Neg Pred Value 1.0000 1.0000 0.9925 0.9819

Prevalence 0.1569 0.1662 0.1754 0.1600

Detection Rate 0.1569 0.1662 0.1692 0.1446

Detection Prevalence 0.1569 0.1662 0.1846 0.1508

Balanced Accuracy 1.0000 1.0000 0.9731 0.9483

Class: PRAD Class: THCA

Sensitivity 1.0000 1.0000

Specificity 1.0000 1.0000

Pos Pred Value 1.0000 1.0000

Neg Pred Value 1.0000 1.0000

```
## Prevalence          0.1692      0.1723
## Detection Rate      0.1692      0.1723
## Detection Prevalence 0.1692      0.1723
## Balanced Accuracy    1.0000      1.0000
```

```
# Random Forest.
set.seed(123)
# Se definen diferentes valores de 'mtry'.
mtry_values <- floor(c(sqrt(ncol(samples_train)),
                        ncol(samples_train)/3, ncol(samples_train)/2))

# Creación de las folds para la validación cruzada.
folds <- createFolds(train_type, k = 5, list = TRUE, returnTrain = FALSE)
# Generación de un vector numérico con la longitud de los valores de 'mtry'
# introducidos.
cv_results <- numeric(length(mtry_values))

# Se genera un vector para almacenar la precisión de cada fold.
for(mtry in seq_along(mtry_values)) {
  fold_accuracies <- numeric(length(folds))

  # Y se realiza la validación cruzada.
  for (fold in seq_along(folds)) {
    validation_idx <- folds[[fold]]
    train_idx <- setdiff(seq_len(nrow(samples_train)), validation_idx)

    # Se realiza el modelo para cada fold.
    rf_fold <- randomForest(samples_train[train_idx, ], train_type[train_idx],
                            mtry = mtry_values[mtry], ntree = 500)
  }
  # Se guarda en 'cv_results' la media de las precisiones obtenidas para cada
# valor de 'mtry'.
  cv_results[mtry] <- mean(fold_accuracies)
}

# Obtención del mejor valor de 'mtry'.
best_mtry <- mtry_values[which.max(cv_results)]

# Se entrena de nuevo el modelo con el valor óptimo obtenido.
fit_rf <- randomForest(samples_train, train_type, mtry = best_mtry,
                       ntree = 500, importance = TRUE)

# Se obtienen las predicciones del conjunto de validación y también sus
# probabilidades.
y_rf_t <- predict(fit_rf, newdata = samples_val)
rf_prob <- predict(fit_rf, newdata = samples_val, type = "prob")

# Se crea e imprime la matriz de confusión a la vez que se obtienen las métricas
# y los valores AUC.
cm_rf_t <- confusionMatrix(validation_type, y_rf_t)
rf_metrics_t <- metrics_ct(cm_rf_t, "Random Forest")
rf_auc <- auc_roc(rf_prob, "Random Forest")
cm_rf_t
```

```
## Confusion Matrix and Statistics
##
##           Reference
```

```

## Prediction BRCA KIRC LUAD LUSC PRAD THCA
##      BRCA    51     0     0     0     0     0
##      KIRC     0    54     0     0     0     0
##      LUAD     0     0    55     2     0     0
##      LUSC     0     0     5    47     0     0
##      PRAD     0     0     0     0    55     0
##      THCA     0     0     0     0     0    56
##
## Overall Statistics
##
##              Accuracy : 0.9785
##              95% CI : (0.9561, 0.9913)
##      No Information Rate : 0.1846
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9741
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: BRCA Class: KIRC Class: LUAD Class: LUSC
## Sensitivity              1.0000      1.0000      0.9167      0.9592
## Specificity              1.0000      1.0000      0.9925      0.9819
## Pos Pred Value           1.0000      1.0000      0.9649      0.9038
## Neg Pred Value           1.0000      1.0000      0.9813      0.9927
## Prevalence               0.1569      0.1662      0.1846      0.1508
## Detection Rate           0.1569      0.1662      0.1692      0.1446
## Detection Prevalence     0.1569      0.1662      0.1754      0.1600
## Balanced Accuracy        1.0000      1.0000      0.9546      0.9705
##
##              Class: PRAD Class: THCA
## Sensitivity              1.0000      1.0000
## Specificity              1.0000      1.0000
## Pos Pred Value           1.0000      1.0000
## Neg Pred Value           1.0000      1.0000
## Prevalence               0.1692      0.1723
## Detection Rate           0.1692      0.1723
## Detection Prevalence     0.1692      0.1723
## Balanced Accuracy        1.0000      1.0000

```

```

# SVM Linear.
set.seed(123)
# Con 'svm()' se entrena el modelo, Con 'cross =' se añade la validación cruzada.
fit_svm_lin <- svm(samples_train, train_type, kernel = "linear", cross = 5,
  probability = TRUE)

# Se realizan las predicciones del conjunto de validación, y se obtienen sus
# probabilidades con 'attr()'.
y_svm_lin <- predict(fit_svm_lin, newdata = samples_val, probability = TRUE)
svm_lin_prob <- attr(y_svm_lin, "probabilities")

# Generación de la matriz de confusión.
cm_svm_lin <- confusionMatrix(validation_type, y_svm_lin)
# Obtención de los estadísticos y valores AUC.
svm_lin_metrics <- metrics_ct(cm_svm_lin, "SVM Linear")
svm_lin_auc <- auc_roc(svm_lin_prob, "SVM Linear")
# Visualización de la matriz de confusión obtenida.

```

```
cm_svm_lin
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction BRCA KIRC LUAD LUSC PRAD THCA
##      BRCA   50    1    0    0    0    0
##      KIRC    0   54    0    0    0    0
##      LUAD    0    0   54    3    0    0
##      LUSC    0    0    4   48    0    0
##      PRAD    0    0    0    0   55    0
##      THCA    0    0    0    0    0   56
##
## Overall Statistics
##
##           Accuracy : 0.9754
##           95% CI : (0.9521, 0.9893)
##      No Information Rate : 0.1785
##      P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.9704
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: BRCA Class: KIRC Class: LUAD Class: LUSC
## Sensitivity           1.0000      0.9818      0.9310      0.9412
## Specificity           0.9964      1.0000      0.9888      0.9854
## Pos Pred Value        0.9804      1.0000      0.9474      0.9231
## Neg Pred Value        1.0000      0.9963      0.9851      0.9890
## Prevalence            0.1538      0.1692      0.1785      0.1569
## Detection Rate        0.1538      0.1662      0.1662      0.1477
## Detection Prevalence  0.1569      0.1662      0.1754      0.1600
## Balanced Accuracy      0.9982      0.9909      0.9599      0.9633
##
##           Class: PRAD Class: THCA
## Sensitivity           1.0000      1.0000
## Specificity           1.0000      1.0000
## Pos Pred Value        1.0000      1.0000
## Neg Pred Value        1.0000      1.0000
## Prevalence            0.1692      0.1723
## Detection Rate        0.1692      0.1723
## Detection Prevalence  0.1692      0.1723
## Balanced Accuracy      1.0000      1.0000

# SVM Radial.

# El código generado es idéntico al SVM lineal, pero en este caso se indica
# "kernel = 'radial'".
set.seed(123)
fit_svm_rad <- svm(samples_train, train_type, kernel = "radial", cross = 5,
                   probability = TRUE)

y_svm_rad <- predict(fit_svm_rad, newdata = samples_val, probability = TRUE)
svm_rad_prob <- attr(y_svm_rad, "probabilities")
```

```
cm_svm_rad <- confusionMatrix(validation_type, y_svm_rad)
svm_rad_metrics <- metrics_ct(cm_svm_rad, "SVM Radial")
svm_rad_auc <- auc_roc(svm_rad_prob, "SVM Radial")
cm_svm_rad
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction BRCA KIRC LUAD LUSC PRAD THCA
```

```
##      BRCA    50     1     0     0     0     0
```

```
##      KIRC     0    54     0     0     0     0
```

```
##      LUAD     0     0    57     0     0     0
```

```
##      LUSC     0     0     5    47     0     0
```

```
##      PRAD     0     0     0     0    55     0
```

```
##      THCA     0     0     0     0     0    56
```

```
##
```

```
## Overall Statistics
```

```
##
```

```
##           Accuracy : 0.9815
```

```
##           95% CI : (0.9603, 0.9932)
```

```
##      No Information Rate : 0.1908
```

```
##      P-Value [Acc > NIR] : < 2.2e-16
```

```
##
```

```
##           Kappa : 0.9778
```

```
##
```

```
##      McNemar's Test P-Value : NA
```

```
##
```

```
## Statistics by Class:
```

```
##
```

```
##           Class: BRCA Class: KIRC Class: LUAD Class: LUSC
```

```
## Sensitivity           1.0000           0.9818           0.9194           1.0000
```

```
## Specificity           0.9964           1.0000           1.0000           0.9820
```

```
## Pos Pred Value        0.9804           1.0000           1.0000           0.9038
```

```
## Neg Pred Value        1.0000           0.9963           0.9813           1.0000
```

```
## Prevalence            0.1538           0.1692           0.1908           0.1446
```

```
## Detection Rate        0.1538           0.1662           0.1754           0.1446
```

```
## Detection Prevalence  0.1569           0.1662           0.1754           0.1600
```

```
## Balanced Accuracy     0.9982           0.9909           0.9597           0.9910
```

```
##           Class: PRAD Class: THCA
```

```
## Sensitivity           1.0000           1.0000
```

```
## Specificity           1.0000           1.0000
```

```
## Pos Pred Value        1.0000           1.0000
```

```
## Neg Pred Value        1.0000           1.0000
```

```
## Prevalence            0.1692           0.1723
```

```
## Detection Rate        0.1692           0.1723
```

```
## Detection Prevalence  0.1692           0.1723
```

```
## Balanced Accuracy     1.0000           1.0000
```

```
# XGBoost.
```

```
set.seed(123)
```

```
# Para poder realizar el XGBoost, los clasificadores se transforman a numérico
```

```
# y se les resta 1 a todos.
```

```
train_type_xgboost <- as.numeric(as.factor(train_type)) - 1
```

```
# Se genera la matriz de trabajo con 'xgb.DMatrix()'.  
dmatrix <-
```

```

xgb.DMatrix(data = samples_train, label = train_type_xgboost)

# Se preparan los parámetros a usar.
params <- list(objective = "multi:softprob",
               num_class = length(unique(train_type_xgboost)),
               max_depth = 6, eta = 0.1, subsample = 0.8,
               colsample_bytree = 0.8, min_child_weight = 1, gamma = 0)

# Realización de la validación cruzada.
cv_xgboost <- xgb.cv(params = params, data = dmatrix, nrounds = 100, nfold = 5,
                    verbose = 0, early_stopping_rounds = 10, prediction = TRUE)

# Entrenamiento del modelo con la mejor iteración obtenida en la validación
# cruzada.
fit_xgb <- xgb.train(params = params, data = dmatrix,
                    nrounds = cv_xgboost$best_iteration, verbose = 0)

# Obtención de las predicciones del conjunto de validación.
xgb_prob <- predict(fit_xgb, newdata = samples_val)
xgb_prob <- matrix(xgb_prob, ncol = length(unique(train_type_xgboost)),
                  byrow = TRUE)
colnames(xgb_prob) <- levels(train_type)

# Se convierten las probabilidades a factor para obtener los clasificadores.
y_xgb_t <- factor(apply(xgb_prob, 1, which.max),
                 levels = 1:length(levels(train_type)),
                 labels = levels(train_type))

# Obtención de la matriz de confusión, las métricas y los valores AUC-ROC.
cm_xgb_t <- confusionMatrix(validation_type, y_xgb_t)
xgb_metrics_t <- metrics_ct(cm_xgb_t, "XGBoost")
xgb_auc <- auc_roc(xgb_prob, "XGBoost")
cm_xgb_t

```

```

## Confusion Matrix and Statistics
##
##               Reference
## Prediction BRCA KIRC LUAD LUSC PRAD THCA
##      BRCA    51     0     0     0     0     0
##      KIRC     0    54     0     0     0     0
##      LUAD     0     0    56     1     0     0
##      LUSC     0     0     3    49     0     0
##      PRAD     0     0     0     0    55     0
##      THCA     0     0     0     0     0    56
##
## Overall Statistics
##
##               Accuracy : 0.9877
##               95% CI : (0.9688, 0.9966)
##      No Information Rate : 0.1815
##      P-Value [Acc > NIR] : < 2.2e-16
##
##               Kappa : 0.9852
##
##      McNemar's Test P-Value : NA
##

```

```
## Statistics by Class:
##
##          Class: BRCA Class: KIRC Class: LUAD Class: LUSC
## Sensitivity          1.0000      1.0000      0.9492      0.9800
## Specificity          1.0000      1.0000      0.9962      0.9891
## Pos Pred Value       1.0000      1.0000      0.9825      0.9423
## Neg Pred Value       1.0000      1.0000      0.9888      0.9963
## Prevalence           0.1569      0.1662      0.1815      0.1538
## Detection Rate       0.1569      0.1662      0.1723      0.1508
## Detection Prevalence 0.1569      0.1662      0.1754      0.1600
## Balanced Accuracy    1.0000      1.0000      0.9727      0.9845
##
##          Class: PRAD Class: THCA
## Sensitivity          1.0000      1.0000
## Specificity          1.0000      1.0000
## Pos Pred Value       1.0000      1.0000
## Neg Pred Value       1.0000      1.0000
## Prevalence           0.1692      0.1723
## Detection Rate       0.1692      0.1723
## Detection Prevalence 0.1692      0.1723
## Balanced Accuracy    1.0000      1.0000
```

```
# Red neuronal convolucional (CNN).
set.seed(123)
# Para poder entrenar la red neuronal, los clasificadores se transforman en
# categóricos.
train_type_cnn <- to_categorical(as.numeric(factor(train_type)) - 1,
                                num_classes = 6)

# Definición de las capas del modelo.
cnn_model_t <- keras_model_sequential() %>%
  layer_dense(units = 64, activation = "relu",
              input_shape = ncol(samples_train)) %>%
  layer_dropout(0.3) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dense(units = 6, activation = "softmax")

# Compilación del modelo.
cnn_model_t %>% compile(loss = "categorical_crossentropy",
                      optimizer = optimizer_adam(),
                      metrics = c("accuracy"))

# Entrenamiento del modelo, aplicando validación interna.
history_cnn <- cnn_model_t %>% fit(samples_train, train_type_cnn, epochs = 50,
                                batch_size = 32, validation_split = 0.2,
                                verbose = 0)

# Se obtienen las predicciones del conjunto de validación.
cnn_prob <- predict(cnn_model_t, samples_val)
```

```
## 11/11 - 0s - 14ms/step
```

```
colnames(cnn_prob) <- levels(train_type)

# Y se transforman estas probabilidades a factor para obtener los clasificadores.
y_cnn_t <- factor(apply(cnn_prob, 1, which.max) - 1,
                 levels = 0:5, labels = levels(train_tumor_type$CancerType))
```



```

# Se obtiene la matriz de confusión y se imprime.
cm_cnn_t <- confusionMatrix(validation_type, y_cnn_t)
# Obtención de los estadísticos y los valores AUC-ROC del modelo.
cnn_metrics_t <- metrics_ct(cm_cnn_t, "CNN")
cnn_auc <- auc_roc(cnn_prob, "CNN")
cm_cnn_t

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction BRCA KIRC LUAD LUSC PRAD THCA
##      BRCA   50    1    0    0    0    0
##      KIRC    0   54    0    0    0    0
##      LUAD    0    0   54    3    0    0
##      LUSC    0    0    2   49    0    1
##      PRAD    0    0    0    0   55    0
##      THCA    0    0    0    0    0   56
##
## Overall Statistics
##
##              Accuracy : 0.9785
##              95% CI : (0.9561, 0.9913)
##      No Information Rate : 0.1754
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9741
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: BRCA Class: KIRC Class: LUAD Class: LUSC
## Sensitivity              1.0000      0.9818      0.9643      0.9423
## Specificity              0.9964      1.0000      0.9888      0.9890
## Pos Pred Value           0.9804      1.0000      0.9474      0.9423
## Neg Pred Value           1.0000      0.9963      0.9925      0.9890
## Prevalence               0.1538      0.1692      0.1723      0.1600
## Detection Rate           0.1538      0.1662      0.1662      0.1508
## Detection Prevalence     0.1569      0.1662      0.1754      0.1600
## Balanced Accuracy        0.9982      0.9909      0.9766      0.9657
##
##              Class: PRAD Class: THCA
## Sensitivity              1.0000      0.9825
## Specificity              1.0000      1.0000
## Pos Pred Value           1.0000      1.0000
## Neg Pred Value           1.0000      0.9963
## Prevalence               0.1692      0.1754
## Detection Rate           0.1692      0.1723
## Detection Prevalence     0.1692      0.1723
## Balanced Accuracy        1.0000      0.9912

```

Para poder escoger qué modelo presenta unos mejores estadísticos a la hora de predecir el tipo de cáncer de las muestras, se ha generado una tabla con los estadísticos básicos obtenidos de la matriz de confusión y otra que contiene el área debajo de la curva ROC para cada una de las clases junto con el valor global.

```
# Se genera un objeto que contiene todas las métricas de los modelos generados.
metrics_models <-
  rbind(knn_metrics, rf_metrics_t, svm_lin_metrics, svm_rad_metrics,
        xgb_metrics_t, cnn_metrics_t)
# Y otro con los valores de AUC-ROC por modelo.
auc_roc_values <- rbind(knn_auc, rf_auc, svm_lin_auc, svm_rad_auc,
                        xgb_auc, cnn_auc)

# Visualización en formato tabla con 'kable()' de las métricas y los AUC-ROC
# de los modelos.
kable(metrics_models,
      caption = "Métricas de los modelos para predecir el tipo de cáncer") %>%
  kable_styling()
```

Table 9: Métricas de los modelos para predecir el tipo de cáncer

Model	Precision	Recall	Specificity	F1_macro	Accuracy	Kappa	Balanced_accuracy
k-NN	0.9793	0.9781	0.9957	0.9785	0.9785	0.9741	0.9869
Random Forest	0.9781	0.9793	0.9957	0.9785	0.9785	0.9741	0.9875
SVM Linear	0.9751	0.9757	0.9951	0.9753	0.9754	0.9704	0.9854
SVM Radial	0.9807	0.9835	0.9964	0.9814	0.9815	0.9778	0.9900
XGBoost	0.9875	0.9882	0.9976	0.9877	0.9877	0.9852	0.9929
CNN	0.9783	0.9785	0.9957	0.9784	0.9785	0.9741	0.9871

```
kable(auc_roc_values, caption = "Valores AUC-ROC para cada tipo de cáncer") %>%
  kable_styling()
```

Table 10: Valores AUC-ROC para cada tipo de cáncer

Model	BRCA	KIRC	LUAD	LUSC	PRAD	THCA	Global
k-NN	1.0000	1	0.9907	0.9597	1	1	0.9917
Random Forest	1.0000	1	0.9952	0.9913	1	1	0.9977
SVM Linear	0.9999	1	0.9973	0.9964	1	1	0.9989
SVM Radial	1.0000	1	0.9976	0.9972	1	1	0.9991
XGBoost	1.0000	1	0.9975	0.9963	1	1	0.9990
CNN	0.9999	1	0.9978	0.9932	1	1	0.9985

Todos los modelos han presentado unas métricas de rendimiento muy altas y unos valores de AUC-ROC muy cercanos a 1, lo que indica que todos son capaces de clasificar de forma robusta las muestras según su tipo de cáncer.

No obstante, entre todos los modelos el que presenta unas mejores métricas es el **XGBoost**, seguido del **SVM Radial**. Estos presentan mejor precisión, *recall* (sensibilidad), especificidad, F1 macro, *accuracy*, Kappa y *accuracy* balanceada que todos los demás modelos.

Cabe destacar que, por los valores de AUC-ROC que presentan todos los modelos, todos ellos han sido capaces de clasificar correctamente todas las muestras de clase KIRC, PRAD y THCA. A su vez, solo **k-NN**, **Random Forest**, **SVM Radial** y **XGBoost** han sido capaces de clasificar todas las muestras clasificadas de tipo BRCA correctamente, mientras que **SVM Lineal** y **CNN** han clasificado erróneamente alguna de estas muestras. Por consiguiente, ninguno de los modelos ha podido clasificar correctamente todas las muestras de tipo LUAD y LUSC; esto se debe principalmente a que ambas se tratan de muestras de cáncer de pulmón, por lo que puede ser que sus niveles de expresión tengan más parecidos que con los demás tipos de cáncer. Para estos dos casos, **SVM Radial** y **XGBoost** son los que han clasificado mejor las muestras, en este orden.

En consecuencia a la información obtenida de las tablas, se ha concluido que los mejores modelos son **SVM Radial** y **XGBoost**. Dado que se trata de una clasificación de 6 clases (son modelos multiclase), se ha optado por **XGBoost**, ya que este permite un mayor ajuste de parámetros y optimización.

Selección, optimización e interpretabilidad del modelo

Modelo de predicción para el tipo de muestra

Para poder optimizar el modelo capaz de identificar muestras normales y tumorales, se ha optado por un proceso de **Random Search** simple generado de forma manual, como se puede observar en el objeto `search`. Con ello, se han generado modelos simultáneamente con diferentes hiperparámetros. De esta forma, el programa es capaz de identificar aquellos valores donde el modelo presenta unos mejores resultados.

A partir de estos hiperparámetros se puede volver a generar el modelo de nuevo, de manera que se espera una mejora en relación a la obtenida previamente durante la generación de los modelos. De nuevo, como se ha hecho antes, se ha generado la matriz de confusión prediciendo las clases de los datos de validación. De esta forma, los estadísticos son comparativos con los obtenidos antes de la optimización.

```
# Preparación de los datos para la optimización del modelo Random Forest.
# Los valores de expresión génica se transforman a matriz, mientras que los
# clasificadores del tipo de muestra se convierten a factor con etiquetas '0'
# y '1'.
samples_train <- as.matrix(train_samples)
train_type <- factor(ifelse(train_cancer_type$SampleType == 2, 1, 0))
samples_val <- as.matrix(validation_samples)
validation_type <- factor(ifelse(validation_cancer_type$SampleType == 2, 1, 0))

set.seed(123)
# Optimización del Random Forest.
train_rf <- data.frame(train_type, samples_train)
validation_rf <- data.frame(validation_type, samples_val)

# Random Search para obtener los mejores hiperparámetros.
search <- data.frame(
  mtry = sample(2:ncol(samples_train), 20, replace = TRUE),
  min.node.size = sample(c(1, 2, 5, 10), 20, replace = TRUE),
  sample.fraction = runif(20, 0.6, 1)
)

# También se añaden los pliegues para generar la validación cruzada.
folds <- createFolds(train_type, k = 5, list = TRUE, returnTrain = FALSE)

# Generación de un dataframe vacío para almacenar los resultados.
results_rf <- data.frame()

for (k in 1:nrow(search)) {
  fold_accuracies <- numeric(length(folds))

  for (fold in seq_along(folds)) {
    val_idx <- folds[[fold]]
    train_idx <- setdiff(seq_len(nrow(train_rf)), val_idx)

    model <- ranger(train_type ~ .,
      data = train_rf[train_idx, ],
      mtry = search$mtry[k],
```

```

        min.node.size = search$min.node.size[k],
        sample.fraction = search$sample.fraction[k],
        probability = TRUE, num.trees = 500,
        classification = TRUE, verbose = FALSE)

    pred_val <- predict(model, train_rf[val_idx, ])$predictions[, 2]
    class_val <- factor(pred_val > 0.5, labels = levels(train_type))

    fold accuracies[fold] <- mean(class_val == train_type[val_idx])
  }

  results_rf <- rbind(results_rf,
                      cbind(search[k, ], Accuracy = mean(fold accuracies)))
}

# Mejores parámetros para realizar el Random Forest.
params_rf <- results_rf[which.max(results_rf$Accuracy), ]

# Generación del nuevo modelo Random Forest optimizado.
rf_final <- ranger(train_type ~ ., data = train_rf,
                  mtry = params_rf$mtry,
                  min.node.size = params_rf$min.node.size,
                  sample.fraction = params_rf$sample.fraction,
                  probability = TRUE, num.trees = 500, classification = TRUE,
                  importance = "impurity")

# Predicción de la clasificación de los datos de validación.
rf_final_prob <- predict(rf_final, validation_rf)$predictions[, 2]
y_rf_final <- factor(rf_final_prob > 0.5, labels = levels(validation_type))

cm_rf_final <- confusionMatrix(validation_type, y_rf_final, positive = "1")
rf_final_metrics <- metrics_samples(cm_rf_final, "RF (opt.)")
cm_rf_final

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0  42    2
##           1   1 324
##
##           Accuracy : 0.9919
##           95% CI : (0.9764, 0.9983)
##           No Information Rate : 0.8835
##           P-Value [Acc > NIR] : 2.831e-16
##
##           Kappa : 0.9609
##
##           Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9939
##           Specificity : 0.9767
##           Pos Pred Value : 0.9969
##           Neg Pred Value : 0.9545
##           Prevalence : 0.8835
##           Detection Rate : 0.8780

```

```
##      Detection Prevalence : 0.8808
##      Balanced Accuracy   : 0.9853
##
##      'Positive' Class    : 1
##
```

Con las métricas de rendimiento obtenidas, se ha generado una tabla que contiene los valores antes y después de la optimización, de forma que se pueda concluir si se ha dado o no una mejora.

```
# Se unen en un mismo objeto las métricas del Random Forest antes y después de
# la optimización.
rf_metrics_comp <- rbind(rf_metrics, rf_final_metrics)

# Comparación de las métricas vista en formato tabla.
kable(rf_metrics_comp,
      caption = "Métricas globales del modelo RF inicial vs optimizado") %>%
kable_styling()
```

Table 11: Métricas globales del modelo RF inicial vs optimizado

Model	Prec_n	Prec_t	Precision	Recall_n	Recall	F1_n	F1_macro	Kappa	Spec	Bal_accc
RF	0.9545	0.9969	0.9757	0.9767	0.9853	0.9655	0.9804	0.9609	0.9767	0.9853
RF (opt.)	0.9545	0.9969	0.9757	0.9767	0.9853	0.9655	0.9804	0.9609	0.9767	0.9853

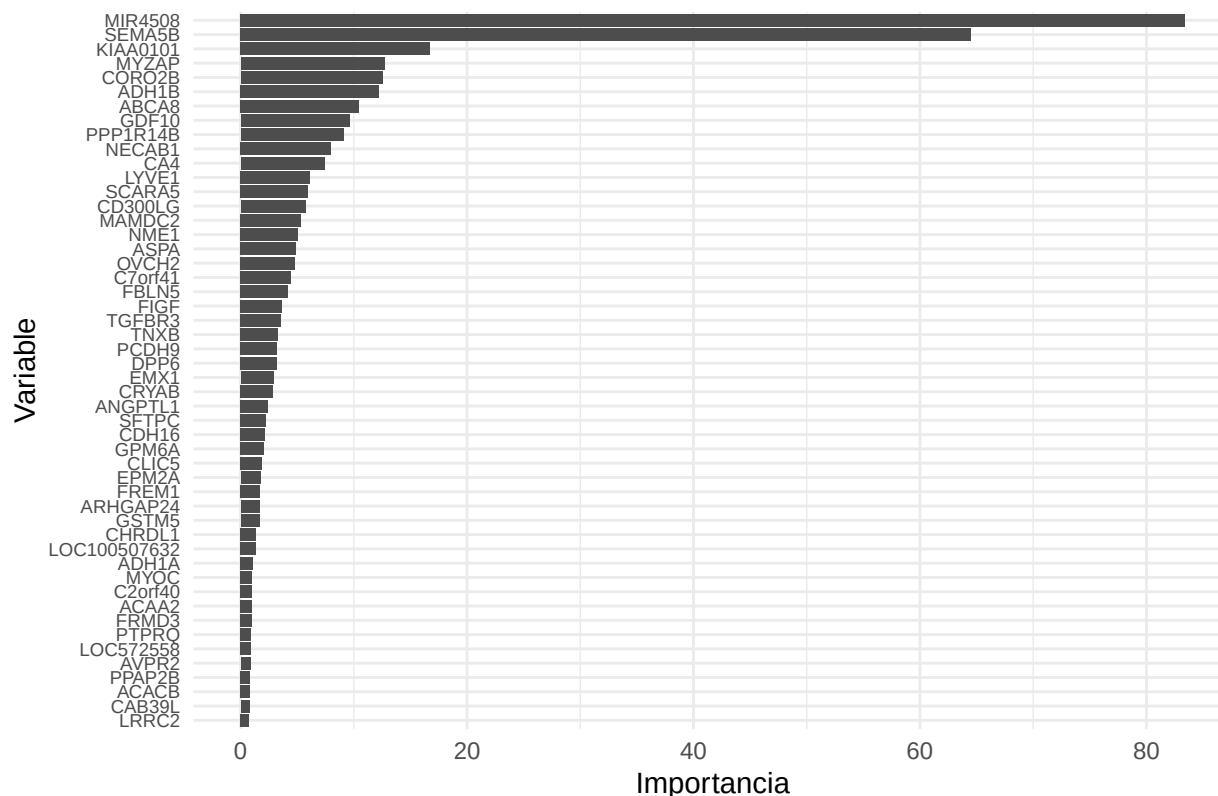
La comparación entre el modelo sin optimizar con el modelo ya optimizado ha mostrado que en ambos casos se obtienen las mismas métricas de rendimiento en el conjunto de validación. Este hecho indica que el modelo inicial ya se encontraba ajustado de manera óptima a los datos, y que el proceso de optimización no ha aportado ninguna mejora adicional. En conclusión, la optimización ha confirmado la robustez y la estabilidad del modelo que se había observado en primera instancia, reforzando así la confianza en la capacidad de generación de este por mucho que no se haya obtenido una mejora en su rendimiento.

```
# Obtención de la importancia de variables del Random Forest optimizado.
importance_rf <- rf_final$variable.importance
# Se ordenan de forma decreciente, obteniendo solo las 50 primeras.
importance_sorted <- sort(importance_rf, decreasing = TRUE)[1:50]

# Generación de un dataframe con las 50 variables más importantes.
df_imp <- data.frame(
  Variable = names(importance_sorted),
  Importance = importance_sorted
)

# Representación gráfica del dataframe.
ggplot(df_imp, aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_bar(stat = "identity", fill = "grey30") +
  coord_flip() +
  labs(title = "Importancia de variables (Random Forest optimizado)",
       x = "Variable", y = "Importancia") + theme_minimal() +
  theme(axis.text.y = element_text(size = 7))
```

Importancia de variables (Random Forest optimizado)



El gráfico de importancia de variables generado ha permitido identificar los genes más contribuyentes a la capacidad predictiva del clasificador. Con este se ha concluido que hay tres que presentan una gran importancia (MIR4508, SEMA5B y KIAA0101), presentando estos unos valores de importancia significativamente superiores a los demás genes. Esta distribución de la importancia indica que el modelo se apoya en una cantidad reducida de variables que contienen mucha información relevante para la clasificación, mientras que muchas otras presentan una contribución más marginal.

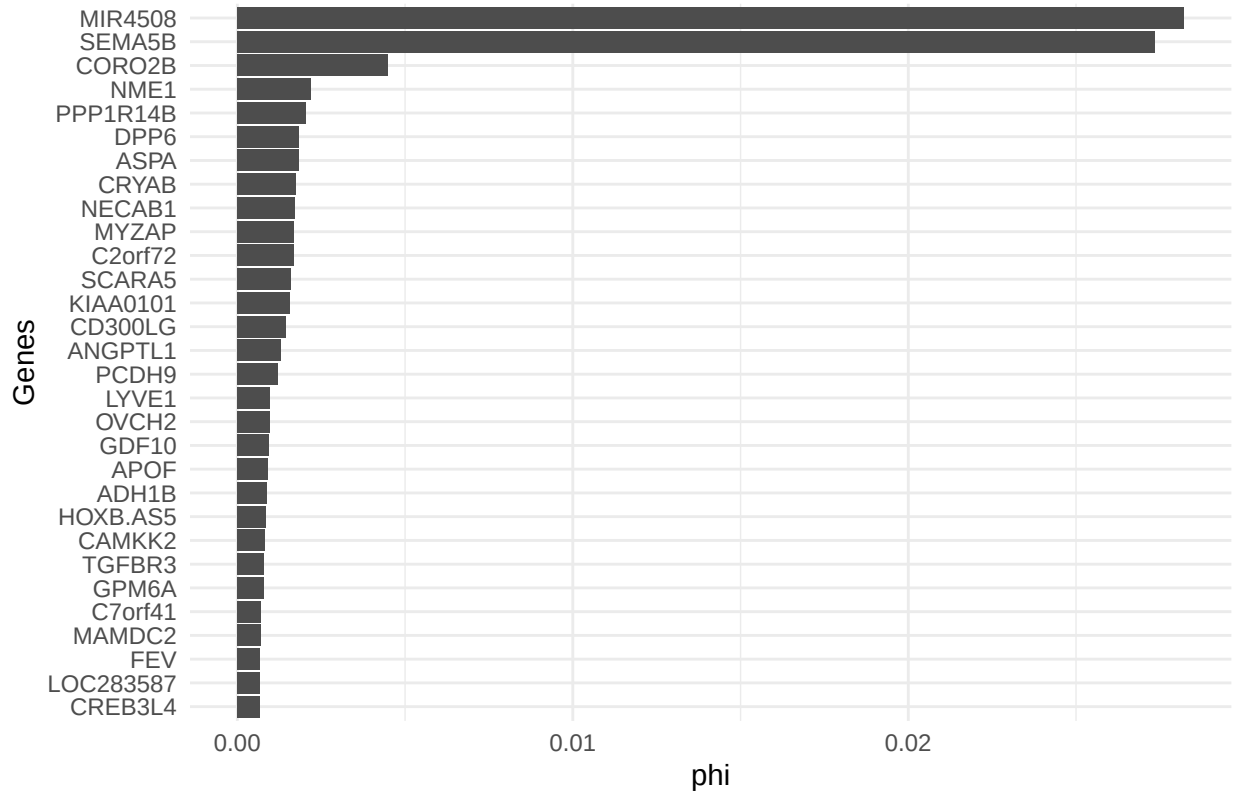
```
predict_rf <- function(model, newdata) {
  # Transformación de los datos a dataframe.
  x <- as.data.frame(newdata)
  # Se añaden columnas faltantes con valor 0.
  missing <- setdiff(colnames(train_rf)[-1], names(x))
  for (m in missing) x[[m]] <- 0
  # Se reordenan las columnas para que coincidan con las del conjunto de datos
  # de entrenamiento.
  x <- x[, colnames(train_rf)[-1], drop = FALSE]

  # Probabilidad de la clase positiva.
  p <- predict(model, x)$predictions[, 2]
  return(p)
}

# Generación de un objeto 'Predictor' a partir del modelo Random Forest.
predictor_rf <- Predictor$new(model = rf_final, data = train_rf[, -1],
                              y = train_rf$train_type,
                              predict.function = predict_rf)

# Se selecciona una muestra del conjunto de validación para explicar.
x_interest_rf <- validation_rf[1, -1, drop = FALSE]
```


Top 30 variables más influyentes (SHAP)



Con este gráfico se ha podido observar que los genes SEMA5B, MIR4508 y, en menor medida, SFTPC han sido aquellos que han presentado los valores SHAP más elevados, indicando una fuerte influencia en la diferenciación entre muestras normales y tumorales. Esta información respalda la obtenida en el gráfico de importancia previo, donde SEMA5B y MIR4508 ya se habían presentado como los genes más importantes.

Modelo de predicción para el tipo de cáncer

```
# Se vuelve a escribir la función de obtención de las AUC-ROC dado que se ha
# observado un problema de solapamiento de funciones con las de otras librerías.
auc_roc <- function(probabilities, model_name) {

  classes <- levels(validation_type)
  auc <- numeric(length(classes))

  for (class in seq_along(classes)) {
    bin_ref <- ifelse(validation_type == classes[class], 1, 0)
    bin_pred <- probabilities[, classes[class]]
    # Se indica pROC::roc() y pROC::auc() para evitar el solapamiento.
    roc <- pROC::roc(bin_ref, bin_pred)
    auc[class] <- pROC::auc(roc)
  }

  names(auc) <- classes

  auc_values <- data.frame("Model" = model_name,
                           "BRCA" = round(unname(auc[1]), 4),
                           "KIRC" = round(unname(auc[2]), 4),
```



```

        "LUAD" = round(unname(auc[3]), 4),
        "LUSC" = round(unname(auc[4]), 4),
        "PRAD" = round(unname(auc[5]), 4),
        "THCA" = round(unname(auc[6]), 4),
        "Global" = round(mean(auc), 4)
    }

```

Finalizada la optimización del modelo para la clasificación del tipo de muestra, se ha procedido con lo mismo pero esta vez para el modelo capaz de identificar el tipo de cáncer al que pertenece cada muestra.

Se ha seleccionado como el mejor modelo y, por tanto, el modelo a optimizar, el generado con **XGBoost**. Sobre este se ha realizado un *Grid Search* (objeto `grid` del código). Con ello, se han obtenido los mejores hiperparámetros para el modelo, pudiendo generar así de nuevo el modelo teniendo en cuenta estos. Tal como se ha hecho antes, se ha predecido de nuevo las clases de los datos de validación y se han obtenido sus estadísticos y sus valores AUC-ROC.

```

# Preparación de los conjuntos de entrenamiento y validación para el proceso de
# optimización del XGBoost para la clasificación del tipo de cáncer.
# Se generan dos objetos con los valores de expresión génica en formato matriz,
# junto con dos otros objetos que contienen solamente los clasificadores del
# tipo de cáncer.
samples_train <- as.matrix(train_tumor_samples)
train_type <- train_tumor_type$CancerType
samples_val <- as.matrix(validation_tumor_samples)
validation_type <- validation_tumor_type$CancerType

# Generación del modelo XGBoost optimizado para la clasificación del tipo de
# cáncer de las muestras.
set.seed(123)
# Se convierten los clasificadores del conjunto de entrenamiento a numérico
# y se le resta 1 a todos.
train_type_xgboost <- as.numeric(as.factor(train_type)) - 1

# Se genera la matriz necesaria en XGBoost con 'xgb.DMatrix()'.
dmatrix <-
  xgb.DMatrix(data = samples_train, label = train_type_xgboost)

# Se realiza un Grid Search para comparar diferentes valores de los
# hiperparámetros.
grid <- expand.grid(eta = c(0.1, 0.3), max_depth = c(4, 6, 8),
  subsample = c(0.8, 1), colsample_bytree = c(0.8, 1))

# Creación de un dataframe vacío para almacenar los resultados.
results_xgb <- data.frame()

for (k in 1:nrow(grid)) {
  # Preparación de los parámetros para cada valor de 'grid'.
  params <- list(objective = "multi:softprob",
    num_class = length(unique(train_type_xgboost)),
    eval_metric = "mlogloss", eta = grid$eta[k],
    max_depth = grid$max_depth[k], subsample = grid$subsample[k],
    colsample_bytree = grid$colsample_bytree[k])

  # Realización de la validación cruzada.
  cv <- xgb.cv(params = params, data = dmatrix, nrounds = 300, nfold = 5,
    early_stopping_rounds = 20, verbose = 0)

```

```

results_xgb <-
  rbind(results_xgb,
        data.frame(eta = grid$eta[k],
                    max_depth = grid$max_depth[k],
                    subsample = grid$subsample[k],
                    colsample_bytree = grid$colsample_bytree[k],
                    best_iter = cv$best_iteration,
                    best_mlogloss =
                      cv$evaluation_log$test_mlogloss_mean[cv$best_iteration]))
}

# Obtención de los mejores hiperparámetros.
best_xgb <- results_xgb[which.min(results_xgb$best_mlogloss), ]

# A partir de los mejores hiperparámetros se realiza el nuevo modelo.
params_xgb <- list(objective = "multi:softprob",
                   num_class = length(unique(train_type_xgboost)),
                   eval_metric = "mlogloss", eta = best_xgb$eta,
                   max_depth = best_xgb$max_depth,
                   subsample = best_xgb$subsample,
                   colsample_bytree = best_xgb$colsample_bytree)

# Entrenamiento final con los mejores parámetros
xgb_final <- xgb.train(params = params_xgb, data = dmatrix,
                      nrounds = best_xgb$best_iter, verbose = 0)

# Se obtiene las probabilidades de las clases del conjunto de validación.
xgb_final_prob <- predict(xgb_final, newdata = samples_val)
xgb_final_prob <-
  matrix(xgb_final_prob, ncol = length(unique(train_type_xgboost)),
        byrow = TRUE)
colnames(xgb_final_prob) <- levels(train_type)

# Y se transforman en factor para obtener los clasificadores.
y_xgb_final <- factor(apply(xgb_final_prob, 1, which.max),
                     levels = 1:length(levels(train_type)),
                     labels = levels(train_type))

# Se obtiene la matriz de confusión, los estadísticos y los AUC-ROC.
cm_xgb_final <- caret::confusionMatrix(validation_type, y_xgb_final)
xgb_final_metrics <- metrics_ct(cm_xgb_final, "XGBoost (opt.)")
xgb_final_auc <- auc_roc(xgb_final_prob, "XGBoost (opt.)")
# Se imprime la matriz de confusión.
cm_xgb_final

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction BRCA KIRC LUAD LUSC PRAD THCA
##      BRCA    51     0     0     0     0     0
##      KIRC     0    54     0     0     0     0
##      LUAD     0     0    56     1     0     0
##      LUSC     0     0     3    49     0     0
##      PRAD     0     0     0     0    55     0
##      THCA     0     0     0     0     0    56
##

```

```
## Overall Statistics
##
##           Accuracy : 0.9877
##           95% CI : (0.9688, 0.9966)
##           No Information Rate : 0.1815
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.9852
##
## Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: BRCA Class: KIRC Class: LUAD Class: LUSC
## Sensitivity           1.0000      1.0000      0.9492      0.9800
## Specificity           1.0000      1.0000      0.9962      0.9891
## Pos Pred Value        1.0000      1.0000      0.9825      0.9423
## Neg Pred Value        1.0000      1.0000      0.9888      0.9963
## Prevalence            0.1569      0.1662      0.1815      0.1538
## Detection Rate        0.1569      0.1662      0.1723      0.1508
## Detection Prevalence  0.1569      0.1662      0.1754      0.1600
## Balanced Accuracy      1.0000      1.0000      0.9727      0.9845
##
##           Class: PRAD Class: THCA
## Sensitivity           1.0000      1.0000
## Specificity           1.0000      1.0000
## Pos Pred Value        1.0000      1.0000
## Neg Pred Value        1.0000      1.0000
## Prevalence            0.1692      0.1723
## Detection Rate        0.1692      0.1723
## Detection Prevalence  0.1692      0.1723
## Balanced Accuracy      1.0000      1.0000
```

Para poder concluir si ha habido o no una mejora en la clasificación, se ha generado una tabla que contiene las métricas de rendimiento del modelo **XGBoost** antes y después de la optimización. También se ha realizado para los valores de AUC-ROC.

```
# Tabla comparativa de las métricas del rendimiento del modelo inicial versus
# el modelo optimizado.
xgb_metrics_comp <- rbind(xgb_metrics_t, xgb_final_metrics)

kable(xgb_metrics_comp,
      caption = "Métricas globales del modelo XGBoost inicial vs optimizado") %>%
kable_styling()
```

Table 12: Métricas globales del modelo XGBoost inicial vs optimizado

Model	Precision	Recall	Specificity	F1_macro	Accuracy	Kappa	Balanced_accuracy
XGBoost	0.9875	0.9882	0.9976	0.9877	0.9877	0.9852	0.9929
XGBoost (opt.)	0.9875	0.9882	0.9976	0.9877	0.9877	0.9852	0.9929

```
# Tabla comparativa de los valores AUC-ROC del rendimiento del modelo inicial
# versus el modelo optimizado.
xgb_auc_comp <- rbind(xgb_auc, xgb_final_auc)
```

```
kable(xgb_auc_comp,
      caption = "AUC-ROC del modelo XGBoost inicial vs optimizado") %>%
      kable_styling()
```

Table 13: AUC-ROC del modelo XGBoost inicial vs optimizado

Model	BRCA	KIRC	LUAD	LUSC	PRAD	THCA	Global
XGBoost	1	1	0.9975	0.9963	1	1	0.9990
XGBoost (opt.)	1	1	0.9973	0.9964	1	1	0.9989

Al comparar las métricas ente el modelo **XGBoost** inicial y la versión optimizada, se ha podido observar que ambos presentan exactamente los mismos valores en las métricas global de precisión, *recall*, especificidad, F1 macro, exactitud global, coeficiente Kappa y exactitud balanceada. Esto indica que, tras la optimización, el rendimiento general del modelo se ha mantenido estable, sin cambios en la capacidad de clasificación ni en la concordancia frente a posibles desbalances de las clases. En este sentido, la optimización no ha supuesto ninguna mejora adicional en los estadísticos globales, pero tampoco ha generado ninguna pérdida en su robustez.

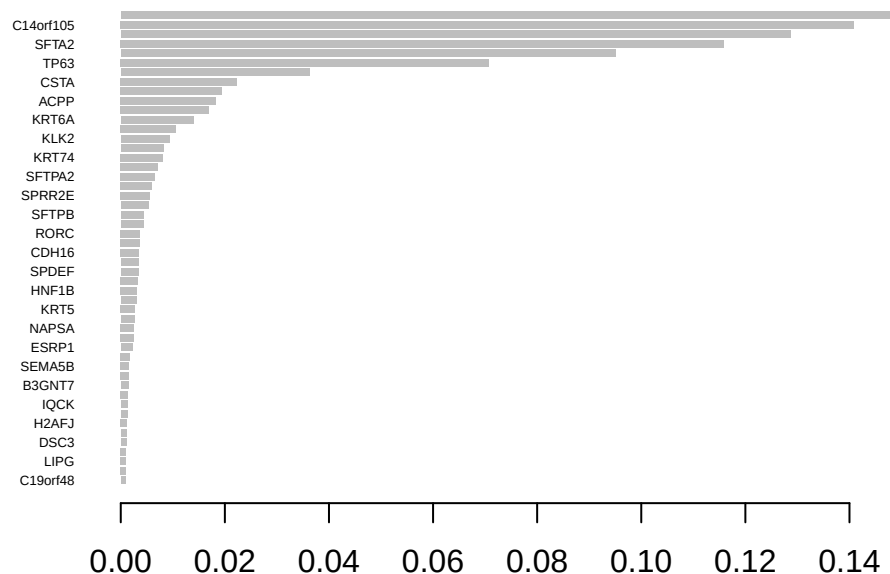
En cuanto al análisis de los valores AUC-ROC, ambos modelos han mostrado un rendimiento muy elevado para todas las clases, alcanzando valores de 1 para las clases BRCA, KIRC, PRAD y THCA. Sin embargo, se han apreciado ligeras variaciones para las clases LUAD y LUSC, donde el modelo inicial ha obtenido valores ligeramente superiores. A nivel global, el AUC-ROC del modelo inicial ha sido ligeramente superior al del modelo optimizado (0.9990 frente a 0.9989, respectivamente). Esto sugiere que, aunque la optimización mantiene el rendimiento de forma casi excelente, implica que haya una pequeña pérdida en la discriminación de estas clases.

En conclusión, el modelo **XGBoost** optimizado ha conservado un rendimiento idéntico al del modelo inicial en los estadísticos, mostrando únicamente variaciones muy leves en los valores AUC-ROC. Esto confirma que la optimización no ha comprometido la calidad del clasificador y que en ambos casos el modelo ha sido capaz de devolver resultados muy confiables para aplicaciones prácticas.

Para interpretar el modelo, se han utilizado dos enfoques diferentes. Primero, se ha analizado la importancia global de las variables (genes) mediante el uso de las funciones `xgb.plot.importance()` y `xgb.importance()`, identificando las 50 más relevantes para la construcción del modelo. Con este tipo de análisis se han podido destacar los biomarcadores que aportan más información para la clasificación de los tipos de cáncer.

También se han usado valores **SHAP** para explicar la predicción de cada una de las muestras. Con esta técnica se ha podido descomponer la contribución individual de cada uno de los genes a la decisión del modelo, mostrando así cuáles han sido las variables que han impulsado o reducido la probabilidad de que esta muestra concreta pertenezca a cada clase.

```
# Se estudia la importancia global de las variables (genes) sobre los datos.
xgb.plot.importance(xgb.importance(model = xgb_final), top_n = 50)
```



```
# Observación de los 10 genes con más importancia.
head(xgb.importance(model = xgb_final), 10)
```

##	Feature	Gain	Cover	Frequency
##	<char>	<num>	<num>	<num>
## 1:	TRPS1	0.14900526	0.055813554	0.021766965
## 2:	C14orf105	0.14082936	0.047298578	0.024327785
## 3:	PAX8	0.12869446	0.036655238	0.008962868
## 4:	SFTA2	0.11592447	0.048512014	0.011523688
## 5:	HOXB13	0.09507290	0.018978315	0.005121639
## 6:	TP63	0.07075882	0.018121320	0.003841229
## 7:	KLK3	0.03623246	0.027960417	0.032010243
## 8:	CSTA	0.02235214	0.009581781	0.002560819
## 9:	C20orf151	0.01948670	0.012447334	0.007682458
## 10:	ACPP	0.01831464	0.008121294	0.003841229

Tal como se ha comentado antes, el gráfico generado mediante las funciones `xgb.plot.importance()` y `xgb.importance()` ha permitido identificar los genes que más contribuyen a la capacidad predictiva del modelo. Para obtener mejor la información sobre los genes más importantes, se han obtenidos también las 10 primeras variables de `xgb.importance()`.

Se puede observar que los genes TRPS1, C14orf105, PAX8 y SFTA2 son los genes que presentan unos valores de importancia más altos (> 0.1). Por ende, estos se podrían considerar como biomarcadores relevantes en la clasificación de distintos tipos de cáncer, ya que el modelo los ha usado para discriminar entre las clases. También presentan unos niveles de importancia elevados los genes HOXB13 y TP63.

Para ampliar la información obtenida, se ha aplicado también un enfoque mediante valores **SHAP**, tal como se observa a continuación. Tal como se ha explicado antes, esta técnica permite descomponer la predicción en contribuciones individuales de cada gen, ofreciendo así una interpretación local del comportamiento del modelo.

```

set.seed(123)
# Se preparan los datos.
feature_names <- colnames(samples_train)
train_shap <- as.data.frame(samples_train)
val_shap <- as.data.frame(samples_val)

# Y se genera la función para Shapley.
predict_xgb <- function(model, newdata) {
  x <- as.data.frame(newdata)
  missing <- setdiff(feature_names, names(x))

  if (length(missing) > 0) {
    for (m in missing) x[[m]] <- 0
  }
  x <- x[, feature_names, drop = FALSE]

  p <- predict(model, as.matrix(x))
  p <- matrix(p, ncol = length(levels(train_type)), byrow = TRUE)
  colnames(p) <- levels(train_type)
  as.data.frame(p)
}

# Se crea el predictor.
predictor <- Predictor$new(model = xgb_final,
                           data = train_shap[, feature_names, drop = FALSE],
                           y = train_type, predict.function = predict_xgb)

x_interest <- val_shap[, feature_names, drop = FALSE]

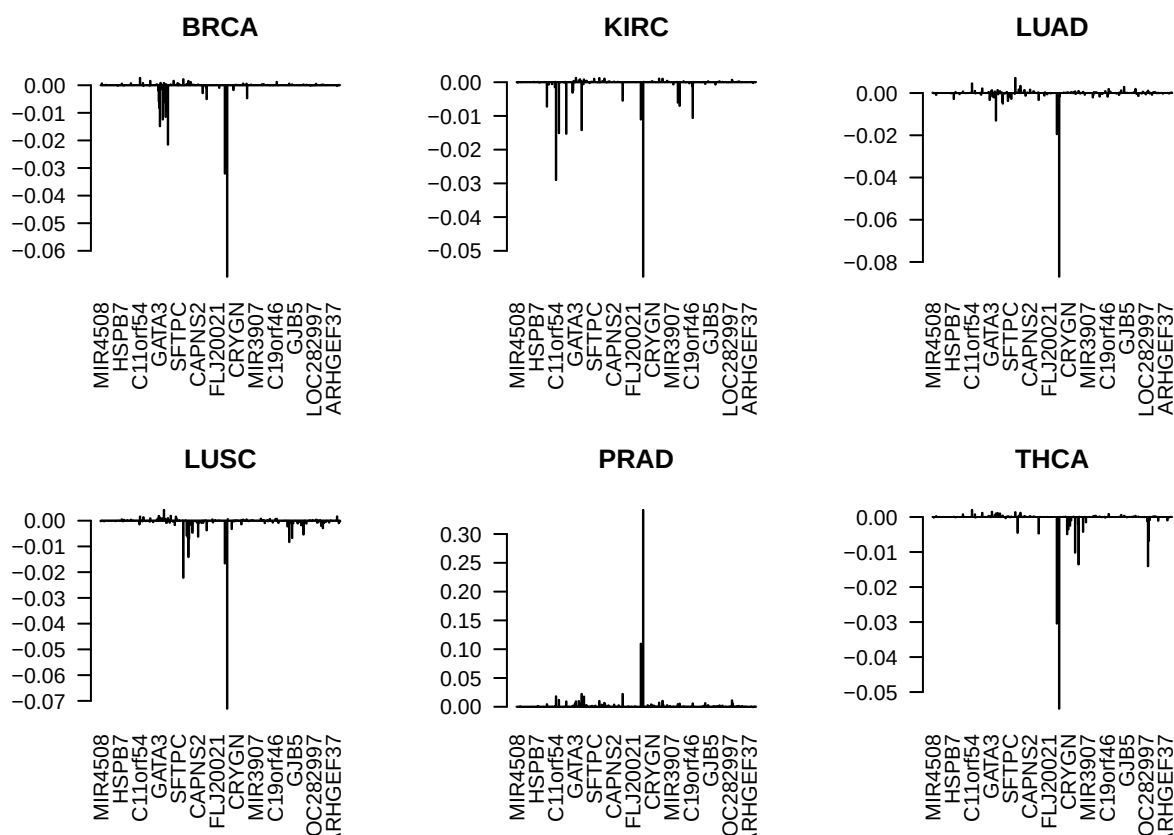
# A través del predictor y la muestra escogida se genera el objeto Shapley.
shap_obj <- Shapley$new(predictor, x_interest = x_interest)

# Obtención de los resultados del SHAP.
shap_df <- shap_obj$results

classes <- levels(train_type)

# Se grafican los resultados para cada clase y se imprimen en pantalla.
par(mfrow = c(2, 3))
for (cls in classes) {
  shap_cls <- subset(shap_df, class == cls)
  barplot(shap_cls$phi, names.arg = shap_cls$feature,
          las = 2, main = cls)
}

```



Como se observa en la imagen superior, se han generado gráficos de barra para cada una de las clases que contiene el modelo. Los valores positivos (barras que tienden a la derecha) indican que el gen favorece la clasificación de la muestra a esa clase concreta, mientras que los valores negativos (barras que tienden hacia la izquierda) disminuyen la probabilidad de clasificación a la clase en cuestión.

En el caso de la muestra analizada, se ha observado que hay una contribución positiva muy marcada hacia la clase PRAD, mientras que todas las demás muestran contribuciones negativas. Esto sugiere que el modelo ha considerado la clase PRAD como la más probable para la muestra que se ha evaluado.

```
# Se indica la clase que se quiere.
cls <- "PRAD"
# Y se realiza un subset sobre los valores de SHAP para obtener los de la clase
# en cuestión.
shap_cls <- subset(shap_df, class == cls)
# Por último, se imprime en pantalla el gen con el valor de 'phi' más alto.
shap_cls[which.max(shap_cls$phi), c("feature", "phi")]
```

```
##      feature      phi
## 1907    KLK3 0.3418983
```

Como se muestra justo en el resultado anterior a este párrafo, KLK3 es el gen que presenta una gran destacación positiva de la muestra a que sea clasificada como PRAD. Con ello se puede concluir que la presencia y el peso de este marcador genético es determinante en la decisión del modelo favoreciendo de forma significativa la clasificación de la muestra hacia este tipo de cáncer. Al mismo tiempo, esta gran influencia reduce la probabilidad de asignación de la muestra a las demás clases, reforzando aún más el papel discriminativo dentro del conjunto de predictores. En conclusión, KLK3 se ha indicado como una variable clave en la capacidad del modelo con tal de distinguir las muestras correspondientes a PRAD.

Prueba final del modelo óptimo

Modelo para la clasificación del tipo de muestra

```
set.seed(123)
# Preparación de los datos para la realización del modelo.
samples_train <- as.matrix(train_samples)
train_type <- factor(ifelse(train_cancer_type$SampleType == 2, 1, 0))
samples_test <- as.matrix(test_samples)
test_type <- factor(ifelse(test_cancer_type$SampleType == 2, 1, 0))

test_rf <- data.frame(test_type, samples_test)

# Obtención de las predicciones del conjunto de prueba.
rf_prob <- predict(rf_final, test_rf)$predictions[, 2]
y_rf <- factor(rf_prob > 0.5, labels = levels(test_type))

# Generación y observación de la matriz de confusión, a la vez que la obtención
# de las métricas de esta.
cm_rf <- confusionMatrix(test_type, y_rf, positive = "1")
rf_metrics <- metrics_samples(cm_rf, "Random Forest")
cm_rf
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 124    2
##           1   3 926
##
##           Accuracy : 0.9953
##           95% CI : (0.989, 0.9985)
##           No Information Rate : 0.8796
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9775
##
##  Mcnemar's Test P-Value : 1
##
##           Sensitivity : 0.9978
##           Specificity : 0.9764
##           Pos Pred Value : 0.9968
##           Neg Pred Value : 0.9841
##           Prevalence : 0.8796
##           Detection Rate : 0.8777
##           Detection Prevalence : 0.8806
##           Balanced Accuracy : 0.9871
##
##           'Positive' Class : 1
##
```

```
# Tabla con las métricas de la prueba final del Random Forest.
kable(rf_metrics,
      caption = "Métricas de la prueba final del Random Forest optimizado") %>%
kable_styling()
```


Table 14: Métricas de la prueba final del Random Forest optimizado

Model	Prec_n	Prec_t	Precision	Recall_n	Recall	F1_n	F1_macro	Kappa	Spec	Bal_acc
Random Forest	0.9841	0.9968	0.9905	0.9764	0.9871	0.9802	0.9888	0.9775	0.9764	0.9871

Modelo para la clasificación del tipo de cáncer

*# Función para la obtención de los valores AUC-ROC actualizada para que
funcione con el conjunto de prueba.*

```
auc_roc <- function(probabilities, model_name) {

  classes <- levels(test_type)
  auc <- numeric(length(classes))

  for (class in seq_along(classes)) {
    bin_ref <- ifelse(test_type == classes[class], 1, 0)
    bin_pred <- probabilities[, classes[class]]
    # Se indica pROC::roc() y pROC::auc() para evitar el solapamiento.
    roc <- pROC::roc(bin_ref, bin_pred)
    auc[class] <- pROC::auc(roc)
  }

  names(auc) <- classes

  auc_values <- data.frame("Model" = model_name,
                           "BRCA" = round(unname(auc[1]), 4),
                           "KIRC" = round(unname(auc[2]), 4),
                           "LUAD" = round(unname(auc[3]), 4),
                           "LUSC" = round(unname(auc[4]), 4),
                           "PRAD" = round(unname(auc[5]), 4),
                           "THCA" = round(unname(auc[6]), 4),
                           "Global" = round(mean(auc), 4))

}
```

```
set.seed(123)
# Preparación de los conjuntos de datos para la realización de la prueba con  
# el modelo XGBoost.
samples_train <- as.matrix(train_tumor_samples)
train_type <- train_tumor_type$CancerType
samples_test <- as.matrix(test_tumor_samples)
test_type <- test_tumor_type$CancerType

# Obtención de las probabilidades de las predicciones sobre el conjunto de datos  
# de prueba.
xgb_prob <- predict(xgb_final, newdata = samples_test)
xgb_prob <- matrix(xgb_prob, ncol = length(unique(train_type_xgboost)),
                  byrow = TRUE)
colnames(xgb_prob) <- levels(train_type)

# Transformación de las probabilidades a factor para la obtención de las  
# predicciones de los clasificadores.
y_xgb <- factor(apply(xgb_prob, 1, which.max),
               levels = 1:length(levels(train_type)),
```

```

labels = levels(train_type))

# Obtención de la matriz de confusión, los estadísticos y los valores AUC-ROC
# de la prueba final.
cm_xgb <- caret::confusionMatrix(test_type, y_xgb)
xgb_metrics <- metrics_ct(cm_xgb, "XGBoost")
xgb_auc <- auc_roc(xgb_prob, "XGBoost")
# Se imprime la matriz de confusión.
cm_xgb

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction BRCA KIRC LUAD LUSC PRAD THCA
##      BRCA  151    1    0    1    0    0
##      KIRC    0   165    0    0    0    0
##      LUAD    0    1   154    5    0    0
##      LUSC    0    0    12   134    0    0
##      PRAD    0    0    0    0   151    0
##      THCA    0    0    0    0    0   154
##
## Overall Statistics
##
##              Accuracy : 0.9785
##              95% CI : (0.9669, 0.9868)
##      No Information Rate : 0.1798
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.9742
##
##      McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##              Class: BRCA Class: KIRC Class: LUAD Class: LUSC
## Sensitivity              1.0000      0.9880      0.9277      0.9571
## Specificity              0.9974      1.0000      0.9921      0.9848
## Pos Pred Value           0.9869      1.0000      0.9625      0.9178
## Neg Pred Value           1.0000      0.9974      0.9844      0.9923
## Prevalence               0.1625      0.1798      0.1787      0.1507
## Detection Rate           0.1625      0.1776      0.1658      0.1442
## Detection Prevalence     0.1647      0.1776      0.1722      0.1572
## Balanced Accuracy         0.9987      0.9940      0.9599      0.9710
##
##              Class: PRAD Class: THCA
## Sensitivity              1.0000      1.0000
## Specificity              1.0000      1.0000
## Pos Pred Value           1.0000      1.0000
## Neg Pred Value           1.0000      1.0000
## Prevalence               0.1625      0.1658
## Detection Rate           0.1625      0.1658
## Detection Prevalence     0.1625      0.1658
## Balanced Accuracy         1.0000      1.0000

```

```

# Visualización en forma de tabla de las métricas obtenidas con la prueba final
# del modelo XGBoost.
kable(xgb_metrics,

```

```
caption = "Métricas de la prueba final del XGBoost optimizado") %>%
kable_styling()
```

Table 15: Métricas de la prueba final del XGBoost optimizado

Model	Precision	Recall	Specificity	F1_macro	Accuracy	Kappa	Balanced_accuracy
XGBoost	0.9779	0.9788	0.9957	0.9782	0.9785	0.9742	0.9873

```
# Visualización en forma de tabla de los valores AUC-ROC obtenidos con la prueba
# final del modelo XGBoost.
```

```
kable(xgb_auc,
      caption = "AUC-ROC de la prueba final del XGBoost optimizado") %>%
kable_styling()
```

Table 16: AUC-ROC de la prueba final del XGBoost optimizado

Model	BRCA	KIRC	LUAD	LUSC	PRAD	THCA	Global
XGBoost	0.9999	1	0.9975	0.9973	1	1	0.9991

Desarrollo de la aplicación web

```
# Se preparan los objetos necesarios para el correcto funcionamiento de la
# aplicación.
selected_genes <- colnames(train_samples)
levels_cancer <- levels(train_type)
# Generación de un fichero '.RData()' en el directorio de trabajo.
save(rf_final, xgb_final, selected_genes, levels_cancer,
     file = "models_with_genes.RData")
```

```
# Carga de los datos anteriores.
load("models_with_genes.RData")

# Se genera una longitud para cada gen, lo cual es necesario para cada gen.
gene_length <- rep(1000, length(selected_genes))
names(gene_length) <- selected_genes

# Tamaño máximo permitido para los archivos subidos.
options(shiny.maxRequestSize = 200*1024^2)

# Interfaz de usuario.
ui <- fluidPage(
  titlePanel("Clasificación jerárquica de muestras normales y tumorales"),

  sidebarLayout(
    sidebarPanel(
      fileInput("file", "Subir muestras",
               accept = c(".csv", ".txt", ".tsv", ".gz")),
      downloadButton("downloadExample", "Descargar archivo de ejemplo"),
      actionButton("predict", "Clasificar")),
```

```

mainPanel(
  tabsetPanel(
    # Panel de la clasificación de las muestras.
    tabPanel("Clasificación",
      h3("Resultados de clasificación"),
      tableOutput("resultsTable"),
      downloadButton("downloadResults", "Descargar resultados")),

    # Panel de ayuda al usuario.
    tabPanel("Requisitos / Ayudas al usuario",
      h3("Formato del archivo"),
      p("La aplicación acepta archivos que se encuentren en formato CSV, TXT, TSV o GZ."),
      p("Cada fila del documento debe corresponder a una muestra, y cada columna a un gen."),
      p("Los valores deben de ser numéricos, exceptuando la columna de los identificadores."),

      h3("Identificador de muestra"),
      p("El archivo puede contener una columna denominada 'Sample' o 'ID' que contenga los id"),
      p("En caso de no existir esta columna, la aplicación asigna nombre de manera automática"),

      h3("Genes requeridos"),
      p("El archivo debe contener todas las columnas correspondientes a los genes utilizados"),
      p("El nombre de los genes debe de aparecer escrito tal y como se muestra en el siguiente archivo"),
      verbatimTextOutput("geneList"),
      p("Los nombres de los genes deben seguir la nomenclatura oficial del",
        a("HUGO Gene Nomenclature Committee (HGNC)",
          href = "https://www.genenames.org/",
          target = "_blank")),

      h3("Número de muestras"),
      p("No hay un límite estricto de muestras, pero archivos muy grandes pueden realizar el"),

      h3("Escala de los datos"),
      p("La aplicación acepta datos crudos o datos normalizados por TPM."),
      p("En caso de que los datos no estén en TPM, la aplicación los normaliza automáticamente"),

      h3("Valores esperados"),
      p("No debe haber valores negativos."),
      p("Valores extremadamente altos pueden indicar un error en el archivo."),

      h3("Archivo de ejemplo"),
      p("Se puede descargar un archivo de ejemplo desde el panel lateral para ver el formato")),

    # Panel con especificaciones técnicas.
    tabPanel("Especificaciones técnicas",
      h3("Modelos utilizados"),
      p("Random Forest entrenado para la clasificación de muestras normales y tumorales."),
      p("XGBoost entrenado para la clasificación de muestras tumorales respecto al tipo de cáncer"),

      h3("Métricas de evaluación del Random Forest"),
      p("Precisión muestras normales: 0.9841"),
      p("Precisión muestras tumorales: 0.9968"),
      p("Precisión global: 0.9905"),
      p("Recall muestras normales: 0.9764"),
      p("Recall global: 0.9871"),
      p("F1-score muestras normales: 0.9802"),
      p("F1-score global: 0.9888"),

```

```

        p("Kappa: 0.9775"),
        p("Sensibilidad: 0.9978"),
        p("Especificidad: 0.9764"),
        p("Balanced Accuracy: 0.9871"),

        h3("Métricas de evaluación del XGBoost"),
        p("Precisión: 0.9779"),
        p("Recall: 0.9788"),
        p("Especificidad: 0.9957"),
        p("F1 macro: 0.9782"),
        p("Accuracy: 0.9785"),
        p("Kappa: 0.9742"),
        p("Balanced Accuracy: 0.9873"),
        shiny::h4("Valores AUC-ROC para cada tipo de cáncer"),
        p("BRCA: 0.9999"),
        p("KIRC: 1"),
        p("LUAD: 0.9975"),
        p("LUSC: 0.9973"),
        p("PRAD: 1"),
        p("THCA: 1"),
        p("Global: 0.9991"),

        h3("Origen de los datos"),
        p("Los datos provienen del estudio 'Alternatively processed and compiled RNA-Sequencing  

          a("GSE62944",
            href = "https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE62944",
            target = "_blank"),"."),
        p("Este estudio, como su nombre indica, trabaja con algunas muestras obtenidas a su vez de")
    )
  )
)

# Función para comprobar si los datos se encuentran en formato TPM.
is_tpm <- function(data, tolerance = 0.20) {
  rs_med <- median(rowSums(data, na.rm = TRUE), na.rm = TRUE)
  pct_diff <- abs(rs_med - 1e6) / 1e6
  return(pct_diff < tolerance)
}

# Función que aplica la normalización TPM.
tpm_normalize <- function(counts, gene_length) {
  rpkm <- counts / (gene_length / 1000)
  tpm <- t(t(rpkm) / colSums(rpkm)) * 1e6
  return(tpm)
}

# Función para comprobar que los datos se encuentran en formato TPM.
ensure_tpm <- function(data, gene_length, tolerance = 0.20) {
  if (is_tpm(data, tolerance)) {
    return(data)
  } else {
    return(tpm_normalize(data, gene_length))
  }
}

```

```

# Servidor.
server <- function(input, output) {

  # Mostrar la lista de los genes requeridos en la ayuda al usuario.
  output$geneList <- renderPrint({
    selected_genes
  })

  # Descarga del archivo de ejemplo.
  output$downloadExample <- downloadHandler(filename = function() {
    "test_samples.csv"
  },
  content = function(file) {
    file.copy("test_samples.csv", file)
  })

  # Evento de clasificación.
  observeEvent(input$predict, {
    req(input$file)

    # Diferentes formas de lectura del archivo según de su extensión.
    ext <- tools::file_ext(input$file$name)
    if (ext == "csv") {
      samples <- fread(input$file$datapath)
    } else if (ext %in% c("txt", "tsv", "gz")) {
      samples <- fread(input$file$datapath, sep = "\t")
    } else {
      showNotification("Formato de archivo no soportado. Usar CSV, TXT o TSV.",
        type = "error")
      return(NULL)
    }

    # Reconocimiento de identificadores de muestra, si es que los hay.
    if ("Sample" %in% colnames(samples)) {
      sample_names <- samples$Sample
      samples[, Sample := NULL]
    } else if ("ID" %in% colnames(samples)) {
      sample_names <- samples$ID
      samples[, ID := NULL]
    } else {
      sample_names <- paste0("Sample ", seq_len(nrow(samples)))
    }

    # Comprobación de que los valores son numéricos.
    if (!all(sapply(samples, is.numeric))) {
      showNotification("El archivo contiene valores no numéricos.",
        type = "error")
      return(NULL)
    }

    # Comprobación de que todos los genes requeridos se encuentran en los datos.
    missing_genes <- setdiff(selected_genes, colnames(samples))
    if (length(missing_genes) > 0) {
      showNotification(
        paste("El archivo no contiene todos los genes necesarios. Faltan: ",
          paste(missing_genes, collapse = ", ")), type = "error",

```

```

    duration = NULL)
  return(NULL)
}

# Normalización TPM de los datos (en caso que sea necesario).
samples_mat <- as.matrix(samples)
samples_mat <- ensure_tpm(samples_mat, gene_length)
samples <- as.data.table(samples_mat)

# Transformación log2(x + 1).
samples <- log2(samples + 1)

# Comprobación de si el archivo contiene valores negativos o demasiado
# elevados.
if (any(samples < 0, na.rm = TRUE)) {
  showNotification("El archivo contiene valores negativos.")
}

if (any(samples > 50, na.rm = TRUE)) {
  showNotification("El archivo contiene valores extremadamente altos.")
}

# Filtrado de los genes para mantener solo los genes necesarios por los
# modelos.
filtered_samples <- samples[, ..selected_genes]
filtered_samples <- as.matrix(filtered_samples)

# Predicciones del modelo Random Forest para clasificar según el tipo de
# muestra.
rf_pred <- predict(rf_final, data.frame(filtered_samples))$predictions
rf_pred <- as.matrix(rf_pred)

if (ncol(rf_pred) == 1) {
  rf_pred <- cbind(1 - rf_pred, rf_pred)
}

rf_prob <- rf_pred[, 2]
rf_classif <- ifelse(rf_prob > 0.5, "Tumoral", "Normal")

prob_rf <- ifelse(rf_classif == "Tumoral", rf_pred[, 2], rf_pred[, 1])

# Inicialización de los resultados del modelo XGBoost.
xgb_classif <- rep("-", nrow(filtered_samples))
cancer_prob <- rep(NA, nrow(filtered_samples))

# Obtención de las muestras tumorales predichas por el Random Forest.
tumor_idx <- which(rf_classif == "Tumoral")

# En caso de que haya alguna, se realiza la predicción sobre estas muestras.
if (length(tumor_idx) > 0) {
  prob <- predict(xgb_final, newdata = filtered_samples[tumor_idx, ])
  prob <- matrix(prob, ncol = length(levels_cancer), byrow = TRUE)
  colnames(prob) <- levels_cancer

  pred_classif <- apply(prob, 1, function(x) colnames(prob)[which.max(x)])
  xgb_classif[tumor_idx] <- pred_classif

```

```

    cancer_prob[tumor_idx] <- apply(prob, 1, max)
  }

  # Generación de un dataframe con los identificadores de muestra, la
  # clasificación según el tipo de muestra y tipo de cáncer y la probabilidad
  # de la clase obtenida en cada caso.
  results <- data.frame(Muestra = sample_names,
                        "Tipo de Muestra" = rf_classif,
                        "Prob. de Muestra" = round(prob_rf, 4),
                        "Tipo de Cáncer" = xgb_classif,
                        "Prob. de Cáncer" = round(cancer_prob, 4),
                        stringsAsFactors = FALSE, check.names = FALSE)

  # Renderizado del dataframe de resultados para la obtención de la tabla
  # en la pantalla de la aplicación.
  output$resultsTable <- renderTable({ results }, rownames = FALSE)

  # Descarga de los resultados en formato CSV.
  output$downloadResults <- downloadHandler(filename = function() {
    paste0("resultados_clasificacion", Sys.Date(), ".csv")
  },
    content = function(file) {
      write.csv(results, file, row.names = FALSE)
    })
})
}

# Lanzamiento de la aplicación.
shinyApp(ui = ui, server = server)

```